

1 Algorithm

1.1 LIS

```

1 // Longest increasing subsequence
2 int LIS(const vector<int>& s) {
3     // use binary search to update the smallest
4     // element
5     // that ends in a subsequence of length d
6     vector<int> v{};
7     v.push_back(s[0]);
8     for (int i{1}; i < s.size(); ++i)
9         if (s[i] > v.back()) v.push_back(s[i]);
10    else *lower_bound(v.begin(), v.end(),
11                    , s[i]) = s[i];
12    return v.size();
13 }
```

1.2 LCS

```

1 // Longest common subsequence
2 int LCS(const vector<int>& a, const vector<
3     int>& b) {
4     vector<vector<int>> dp(a.size() + 1,
5         vector<int>(b.size() + 1));
6     for (int i{1}; i <= int(a.size()); ++i)
7         for (int j{1}; j <= int(b.size()); ++j) {
8             dp[i][j] = max(dp[i][j], max(dp[i - 1][j], dp[i][j - 1]));
9             if (a[i - 1] == b[j - 1]) dp[i][j] = max(dp[i][j], dp[i - 1][j - 1] + 1);
10        }
11    return dp[a.size()][b.size()];
12 }
```

2 Basic

2.1 mod helper function

```

1 int add(int i, int j) {
2     if ((i += j) >= MOD)
3         i -= MOD;
4     return i;
5 }
6
7 int sub(int i, int j) {
8     if ((i -= j) < 0)
9         i += MOD;
10    return i;
11 }
```

2.2 self-defined-pq-operator

```

1 auto cmp = [](int a, int b) {
2     return a > b;
3 };
4 priority_queue<int, vector<int>, decltype(
5     cmp)> pq(cmp);
```

2.3 generating all subsets

```

1 for (int b = 0; b < (1<<n); b++) {
2     vector<int> subset;
3     for (int i = 0; i < n; i++) {
4         if (b&(1<<i)) subset.push_back(vc[i]);
5     }
6 }
```

2.4 tuple

```

1 int main() {
2     tuple<int, double, string> myTuple(100,
3         3.14, "hello world");
4     tuple<int, double, string> anotherTuple
5         = make_tuple(100, 18, "Tom");
6     cout << get<0>(myTuple) << "\n"; // 取得
7         第一個元素
8     auto [a, b, c] = myTuple;
9     get<1>(myTuple) = 5.43; // 修改第二個元
10        素的值
11
12     int a;
13     double b;
14     string c;
15     tie(a, b, c) = myTuple;
16 }
```

2.5 memset

```

1 memset(a, 0, sizeof(a)); // 0
2 memset(a, 0x3f3f3f3f, sizeof(a)); // INF
```

2.6 submask enumeration

```

1 // O(3^n)
2
3 int m = 0b10110; // binary: 10110, decimal:
4     22
5 cout << "Mask: " << bitset<5>(m) << " (" <<
6     m << ")\n";
```

```

7 // Enumerate all submasks
8 for (int s = m; s; s = (s - 1) & m) {
9     cout << bitset<5>(s) << " (" << s << ")\n";
10 }
11
12 // Optionally include the empty submask
13 cout << bitset<5>(0) << " (0)\n";
```

2.7 stringstream split by comma

```

1 while (std::getline(ss, segment, ',')) {
2     segments.push_back(segment);
3 }
```

3 DP

3.1 deque

```

1 /*
2 遊戲 DP - O(N^2)
3 A 與 B 將進行以下的遊戲。
4
5 最初，他們會得到一個序列 a = (a1, a2, ...,
6     aN)。在 a 尚未為空時，兩位玩家輪流進行以
7     下操作，從 A 開始：
8
9     從 a 的開頭或結尾移除一個元素。玩家會獲得 x
10    分，其中 x 為被移除的元素。
11
12    設 X 與 Y 分別為遊戲結束時 A 與 B 的總得分。
13    A 會嘗試最大化 X-Y，而 B 會嘗試最小化 X-Y。
14
15    假設兩位玩家都採取最優策略，請求出最後的 X-Y
16    值。
17
18    定義 dp[i][j] 為在區間 [i, j] 上，對於 B 來
19    說的最優分數 (X-Y)。
20
21    */
22 void solve() {
23     int n;
24     cin >> n;
25     vector<int> a(n);
26     vector<vector<int>> dp(n + 1, vector<int>
27         (n + 1, 0));
28
29     for (int i = 0; i < n; ++i) {
30         cin >> a[i];
31         dp[i][i] = a[i];
32     }
33
34     for (int i = n - 1; i >= 0; --i) {
35         for (int j = i + 1; j < n; ++j) {
36             dp[i][j] = max(a[i] - dp[i + 1][j], a[j] - dp[i][j + 1]);
37         }
38     }
39 }
```

```

29     }
30 }
31 cout << dp[0][n - 1] << "\n";
32 }
```

3.2 walk

```

1 /*
2 DP on graphs - O(N^3 Log K)
3 給定一個簡單的有向圖 G，具有 N 個頂點，編號
4     為 1, 2, ..., N。
5
6 對於任意 i, j (1 ≤ i, j ≤ N)，給定整數 a_{i,j}，表示是否存在從頂點 i 指向頂點 j 的有
7     向邊。若 a_{i,j} = 1，則存在邊；若 a_{i,j} = 0，則不存在。
8
9 求圖中長度為 K 的不同有向路徑數目，對 10^9+7
10    取模。路徑可重複通過相同邊（即允許重複
11    邊）。
12
13 注意：當我們將鄰接矩陣 m 與 m 相乘時，得到的是
14     長度為 2 的路徑數；若取 m 的 p 次方 m^p，
15     則其 (i, j) 元素表示從 i 到 j 的長度
16     為 p 的路徑數。
17
18    */
19 void solve() {
20     int n, k;
21     cin >> n >> k;
22     vector<vector<int>> m(n, vector<int>(n));
23
24     for (int i = 0; i < n; ++i) {
25         for (int j = 0; j < n; ++j) {
26             cin >> m[i][j];
27         }
28     }
29
30     Matrix<int> mat(m);
31     mat = power(mat, k);
32     int ans = 0;
33     for (int i = 0; i < n; ++i) {
34         for (int j = 0; j < n; ++j) {
35             ans += mat.mat[i][j];
36             ans %= MOD;
37         }
38     }
39     cout << ans << "\n";
40 }
```

3.3 grouping

```

1  /*
2  狀態  $DP - O(3^N * 2^N * N^2)$ 
3  有  $N$  隻兔子，編號為  $1, 2, \dots, N$ 。
4
5  對於每一對  $i, j$  ( $1 \leq i, j \leq N$ )，兔子  $i$  與  $j$  的相容
   度由整數  $a_{i,j}$  描述。這裡  $a_{i,i} = 0$  對於
   每個  $i$  ( $1 \leq i \leq N$ )，且  $a_{i,j} = a_{j,i}$  對於任
   意  $i$  與  $j$  ( $1 \leq i, j \leq N$ )。
6
7  A 將  $N$  隻兔子分成若干個群組。每隻兔子必須且
   僅屬於一個群組。分群後，對於每一對  $i$  與
    $j$  ( $1 \leq i < j \leq N$ )，若兔子  $i$  與  $j$  屬於同一群
   組，A 即可獲得  $a_{i,j}$  分。
8
9  求 A 能獲得的最大總分。
10
11 令  $cost[S]$  表示將集合  $S$  中的所有兔子放在同一
   群組時所得到的分數。此值可在  $O(2^N * N^2)$  時間內計算。
12
13 接著我們計算  $dp[S]$ ，表示對集合  $S$  中的兔子進
   行分群時所能得到的最大分數。
14
15 */
16 void solve() {
17     int n;
18     cin >> n;
19     vector<vector<int>>> a(n, vector<int>(n));
20     ;
21     vector<int> cost(1<<n, 0);
22     vector<int> dp(1<<n, 0);
23
24     for (int i = 0; i < n; ++i) {
25         for (int j = 0; j < n; ++j) {
26             cin >> a[i][j];
27         }
28     }
29
30     // backtrack all subset
31     for (int b = 0; b < (1<<n); ++b) {
32         vector<int> subset;
33         for (int i = 0; i < n; ++i) {
34             if (b & (1<<i)) {
35                 for (const int& j : subset) {
36                     cost[b] += a[i][j];
37                 }
38             }
39         }
40     }
41
42     // dp
43     for (int i = 0; i < (1<<n); ++i) {
44         int j = ((1<<n) - 1) ^ i;
45         for (int s = j; s != 0; s = (s - 1)
46             & j) {
47             dp[i ^ s] = max(dp[i ^ s], dp[i]
48                 + cost[s]);
49         }
50     }

```

3.4 matching

```

48     }
49     cout << dp[(1<<n) - 1] << "\n";
50 }

```

/*
Bitmask DP - $O(N * 2^N)$
有 N 個男人和 N 個女人，分別編號為 $1, 2, \dots, N$ 。
對於每個 i, j ($1 \leq i, j \leq N$)，男人 i 和女人 j 的相容性由整數 $a[i][j]$ 給出。
如果 $a[i][j] = 1$ ，則男人 i 和女人 j 是相容的；
如果 $a[i][j] = 0$ ，則不是。
A 正在嘗試組成 N 對，每對由一個相容的男人和女人組成。在這裡，每個男人和每個女人必須恰好屬於一對。
求 A 可以組成 N 對的方法數，結果對 $10^9 + 7$ 取模。
定義 $dp[S]$ 為將集合 S 中的女性與前 $|S|$ 個男性配對的方法數。
*/
const int maxn = 21;
const int MOD = 1e9 + 7;
int n;
int grid[maxn][maxn];
int dp[1<<maxn];
void solve() {
 cin >> n;
 memset(dp, 0, sizeof(dp));
 for (int i = 0; i < n; ++i) {
 for (int j = 0; j < n; ++j) {
 cin >> grid[i][j];
 }
 }
 dp[0] = 1;
 for (int s = 0; s < (1<<n); ++s) {
 int ps = __builtin_popcount(s);
 for (int w = 0; w < n; ++w) {
 if ((s & (1<<w)) || !grid[ps][w]) {
 continue;
 }
 dp[s | (1<<w)] += dp[s];
 dp[s | (1<<w)] %= MOD;
 }
 }
 cout << dp[(1<<n) - 1] << "\n";
}

3.5 projects

```

1  /*
2  LIS DP -  $O(N \log N)$ 
3  有  $n$  個你可以參加的專案。對於每個專案，你知
   道其開始與結束天數以及可獲得的報酬金額。
4  在同一天你最多只能參加一個專案。
5  問：你最多可以賺到多少金額？
6
7   $dp[i]$  = 在第  $i$  天之前我們可以賺到的最大金
   額。
8  */
9
10 void solve() {
11     int n;
12     cin >> n;
13     vector<array<int, 3>>> vc(n);
14     map<int, int> days;
15
16     for (int i = 0; i < n; ++i) {
17         int a, b, p;
18         cin >> a >> b >> p;
19         days[a] = days[b] = 1;
20         vc[i] = {a, b, p};
21     }
22     int idx = 1;
23     for (auto& x : days) {
24         x.second = idx++;
25     }
26     vector<int> dp(idx, 0);
27
28     sort(vc.begin(), vc.end(), [](const
29         array<int, 3>& va, const array<int,
30         3>& vb) {
31         if (va[1] != vb[1]) return va[1] <
32             vb[1];
33         if (va[0] != vb[0]) return va[0] <
34             vb[0];
35         return va[2] > vb[2];
36     });
37
38     int i = 0;
39     for (int d = 1; d < idx; ++d) {
40         dp[d] = dp[d - 1];
41         while (i < n && days[vc[i][1]] == d) {
42             dp[d] = max(dp[d], dp[days[vc[i][0]] - 1] + vc[i][2]);
43             i++;
44         }
45     }
46     cout << dp[idx - 1] << "\n";
47 }

```

3.6 stones

```

1  /*
2  遊戲  $DP - O(N^2)$ 
3  有一個集合  $A = \{a_1, a_2, \dots, a_N\}$ ，包含  $N$  個正整
   數。太郎和次郎將進行以下的遊戲。
4

```

```

5 一開始有一堆  $K$  顆石頭。兩位玩家輪流進行以下
   操作，從太郎開始：
6
7 選擇集合  $A$  中的一個元素  $x$ ，並從石堆中移除恰
   好  $x$  顆石頭。
8 當某位玩家無法進行操作時即輸掉比賽。假設兩位
   玩家都採取最優策略，請判斷誰會獲勝。
9
10 定義  $dp[i]$  表示當剩下  $i$  顆石頭時，是否有可能
   獲勝。
11
12 */
13 void solve() {
14     int n, k;
15     cin >> n >> k;
16     vector<int> a(n);
17     vector<bool> dp(k + 1, 0);
18
19     for (int i = 0; i < n; ++i) {
20         cin >> a[i];
21     }
22
23     for (int i = 1; i <= k; ++i) {
24         for (int x : a) {
25             if (i >= x && !dp[i - x]) {
26                 dp[i] = 1;
27             }
28         }
29     }
30     cout << (dp[k] ? "First" : "Second") <<
31         "\n";
32 }

```

3.7 coins

```

1  /*
2  機率  $DP - O(N^2)$ 
3  給定一個正奇數  $N$ 
4  有  $N$  枚編號為  $1, 2, \dots, N$  的硬幣，第  $i$  枚出現正
   面的機率為  $p$ ，反面為  $1 - p$ 。
5  已經拋擲所有硬幣，求正面數大於反面的機率。
6
7  定義  $dp[i][j]$  為拋完前  $i$  枚硬幣後，得到  $j$  次
   正面的機率。
8  */
9
10 void solve() {
11     int n;
12     cin >> n;
13     vector<double> a(n);
14     vector<vector<double>>> dp(n + 1, vector<
15         double>(n + 1, 0.0));
16
17     for (int i = 0; i < n; ++i) {
18         cin >> a[i];
19     }
20
21     for (int i = 0; i <= n; ++i) {
22         dp[i][0] = 1.0;
23     }

```

```

24 int least = n / 2 + 1;
25
26 for (int i = 1; i <= n; ++i) {
27     for (int j = 1; j <= least; ++j) {
28         // head
29         dp[i][j] = dp[i - 1][j - 1] * a[
30             i - 1];
31         // tail
32         dp[i][j] += dp[i - 1][j] * (1 -
33             a[i - 1]);
34     }
35 }
36
37 cout << fixed << setprecision(10) << dp[
38     n][least] << "\n";

```

3.8 elevator rides

```

1 /*
2 狀態 DP -  $O(2^N)$ 
3 有  $n$  個人想要搭電梯到樓頂。建築物只有一部電
4 梯。你知道每個人的體重以及電梯的最大允許
5 載重。最少需要搭乘多少次電梯？
6 */
7
8 void solve() {
9     int n, x;
10    cin >> n >> x;
11    vector<int> w(n);
12    vector<pii> dp(1<n, {INF, INF});
13
14    for (int i = 0; i < n; ++i) {
15        cin >> w[i];
16    }
17
18    dp[0] = {1, 0};
19    for (int b = 1; b < (1<n); ++b) {
20        for (int i = 0; i < n; ++i) {
21            if (b & (1<i)) {
22                auto [r_prev, w_prev] = dp[b
23                    ^ (1<i)];
24                pii can;
25                if (w_prev + w[i] <= x) {
26                    can = {r_prev, w_prev +
27                        w[i]};
28                }
29                else {
30                    can = {r_prev + 1, w[i
31                        ]};
32                }
33                dp[b] = min(dp[b], can);
34            }
35        }
36    }
37    cout << dp[(1<n) - 1].first << "\n";
38 }

```

3.9 slimes

```

1 /*
2 Range DP -  $O(N^3)$ 
3 有  $N$  個史萊姆排成一列。最初，從左邊數來第  $i$ 
4 個史萊姆的大小為  $a_i$ 。
5 A 想要把所有史萊姆合併成一個更大的史萊姆。他
6 會重複執行以下操作，直到只剩下一個史萊姆
7 為止：
8 選擇兩個相鄰的史萊姆，將它們合併成一個新的史
9 萊姆。新史萊姆的大小為  $x+y$ ，其中  $x$  和  $y$ 
10 是合併前兩個史萊姆的大小。
11 這時會產生  $x+y$  的花費。合併時，史萊姆的相對
12 位置不會改變。
13 請求出合併所有史萊姆所需的最小總花費。
14
15 令  $dp[i][j]$  表示將第  $i$  個到第  $j$  個史萊姆合併
16 成一個史萊姆的最小花費。
17 */
18
19 const int maxn = 401;
20 const int INF = 1e18;
21 int dp[maxn][maxn];
22 int a[maxn];
23 int prefix[maxn + 1];
24
25 int f(int i, int j) {
26     if (i + 1 == j) {
27         return a[i] + a[j];
28     }
29     if (i == j) {
30         return 0;
31     }
32     if (dp[i][j] != INF) {
33         return dp[i][j];
34     }
35     //cerr << i << " " << j << "\n";
36
37     int ans = INF;
38     for (int k = i; k < j; ++k) {
39         ans = min(ans, f(i, k) + f(k + 1, j)
40             );
41     }
42     return dp[i][j] = ans + (prefix[j + 1] -
43         prefix[i]);
44 }

```

3.10 digit sum

```

1 /*
2 Digit DP -  $O(|K| * D)$ 
3 計算在  $1$  到  $K$  (含) 之間，滿足其十進位數字和
4 為  $D$  的倍數的整數數量。答案對  $10^9+7$  取
5 模。
6
7 令  $dp[i][j]$  表示在已確定前  $i$  位數字的情況
8 下，構成長度為  $|K|$  的數字且目前數字和

```

```

1 mod  $D$  等於  $j$  的方法數。
2 */
3
4 const int MOD = 1e9 + 7;
5 int dp[10001][101][2];
6
7 void solve() {
8     string K;
9     int D;
10    cin >> K >> D;
11    int len = K.size();
12    memset(dp, 0, sizeof(dp));
13    dp[0][0][1] = 1;
14
15    for (int i = 1; i <= len; ++i) {
16        int limit = K[i - 1] - '0';
17        for (int s = 0; s < D; ++s) {
18            for (int flag = 0; flag <= 1; ++
19                flag) {
20                int ways = dp[i - 1][s][flag
21                    ];
22                if (ways == 0) continue;
23                int max_d = (flag ? limit :
24                    9);
25                for (int d = 0; d <= max_d;
26                    ++d) {
27                    int rs = (s + d) % D;
28                    int rflag = (flag && d
29                        == max_d ? 1 : 0);
30                    dp[i][rs][rflag] += ways
31                        ;
32                    dp[i][rs][rflag] %= MOD;
33                }
34            }
35        }
36    }
37
38    int ans = (dp[len][0][0] + dp[len
39        ][0][1]) % MOD;
40    ans = (ans - 1 + MOD) % MOD;
41    cout << ans << "\n";
42 }

```

3.11 sushi

```

1 /*
2 期望值 DP -  $O(N^3)$ 
3 有  $N$  盤壽司，編號從  $1$  到  $N$ 。第  $i$  盤最初有  $a_i$ 
4 ( $1 \leq a_i \leq 3$ ) 塊壽司。
5 太郎不斷擲一顆編號  $1$  到  $N$  的骰子。如果結果是
6 第  $i$  盤且該盤還有壽司，他就吃掉一塊；否
7 則什麼也不做。
8
9 請求出吃完所有壽司所需擲骰子的期望次數。
10
11  $dp[x][y][z]$  代表還有
12  $x$  盤剩 1 塊壽司、
13  $y$  盤剩 2 塊壽司、
14  $z$  盤剩 3 塊壽司時的期望擲骰次數。
15 */

```

```

14 const int maxn = 301;
15 double dp[maxn][maxn][maxn];
16 int n;
17
18 double dfs(int x, int y, int z) {
19     if (x < 0 || y < 0 || z < 0) return 0;
20     if (x == 0 && y == 0 && z == 0) return
21         0;
22     if (dp[x][y][z] > 0) return dp[x][y][z];
23     double ans = n + x * dfs(x - 1, y, z)
24         + y * dfs(x + 1, y - 1, z
25             )
26         + z * dfs(x, y + 1, z -
27             1);
28     return dp[x][y][z] = ans / (x + y + z);
29 }
30
31 void solve() {
32     cin >> n;
33     vector<int> a(n);
34     memset(dp, -1, sizeof(dp));
35     vector<int> freq(4, 0);
36
37     for (int i = 0; i < n; ++i) {
38         cin >> a[i];
39         freq[a[i]]++;
40     }
41
42     cout << fixed << setprecision(10) << dfs
43         (freq[1], freq[2], freq[3]) << "\n";
44 }

```

3.12 candies

```

1 /*
2 組合 DP -  $O(NK)$ 
3 有  $N$  個小孩，編號為  $1, 2, \dots, N$ 。
4
5 他們決定將  $K$  顆糖果分給自己。對於每個  $i$  ( $1 \leq i \leq N$ )，第  $i$  個小孩最多可以拿到  $a_i$  顆糖果
6 (包含  $0$  顆)。所有糖果都必須分完，不能
7 剩下。
8
9 請問有多少種分配糖果的方法？請將答案對
10  $10^9+7$  取模。若存在某個小孩分到的糖果數
11 不同，則視為不同的分配方式。
12
13 令  $dp[i][j]$  表示將  $j$  顆糖果分給前  $i$  個小孩的
14 方法數。
15 */
16
17 void solve() {
18     int n, k;
19     cin >> n >> k;
20     vector<int> a(n);
21     vector<int> dp(k + 1, 0), S(k + 1, 0);
22
23     for (int i = 0; i < n; ++i) {
24         cin >> a[i];
25     }
26 }

```

```

21 dp[0] = 1;
22 for (int i = 0; i < n; ++i) {
23     vector<int> new_dp(k + 1, 0);
24     S[0] = dp[0];
25     for (int j = 1; j <= k; ++j) {
26         S[j] = (S[j - 1] + dp[j]) % MOD;
27     }
28     for (int j = 0; j <= k; ++j) {
29         if (j - a[i] - 1 >= 0) {
30             new_dp[j] = (S[j] - S[j - a[i] - 1] + MOD) % MOD;
31         }
32         else {
33             new_dp[j] = S[j] % MOD;
34         }
35     }
36     dp = new_dp;
37 }
38 cout << dp[k] << "\n";
39 }
40 }

```

3.13 permutation

```

1  /*
2  抽象 DP -  $O(N^2)$ 
3  設  $N$  為正整數。給定一個長度為  $N-1$  的字串  $s$ ，
4  字元僅包含 ' $<$ ' 與 ' $>$ '。
5  求滿足條件的排列 ( $p_1, p_2, \dots, p_N$ ) (即 1 到  $N$ 
6  的排列) 數量。答案對  $10^9+7$  取模：
7  對於每個  $i$  ( $1 \leq i \leq N-1$ )，若  $s$  的第  $i$  個字
8  元為 ' $<$ '，則要求  $p_i < p_{i+1}$ ；若為 ' $>$ '，
9  則要求  $p_i > p_{i+1}$ 。
10 */
11 void solve() {
12     int n;
13     string s;
14     cin >> n >> s;
15     vector<vector<int>> dp(n + 1, vector<int>
16         >(n + 1, 0));
17     vector<int> prefix(n + 1, 0);
18     dp[1][0] = 1;
19
20     for (int i = 2; i <= n; ++i) {
21         for (int k = 0; k < n; ++k) {
22             prefix[k + 1] = prefix[k] + dp[i - 1][k];
23         }
24         for (int j = 0; j < i; ++j) {
25             if (s[i - 2] == '>') {
26                 dp[i][j] += prefix[i - 1] - prefix[j];
27                 dp[i][j] %= MOD;
28             }
29             else {
30                 dp[i][j] += prefix[j];
31                 dp[i][j] %= MOD;
32             }
33         }
34     }
35     cout << dp[n][n - 1] << "\n";
36 }

```

```

29     for (int k = j; k < i - 1;
30         ++k) {
31         dp[i][j] += dp[i - 1][k];
32     }
33     }
34     else {
35         dp[i][j] += prefix[j];
36         dp[i][j] %= MOD;
37     }
38     for (int k = 0; k < j; ++k) {
39         dp[i][j] += dp[i - 1][k];
40     }
41     }
42     }
43     }
44     }
45     int ans = 0;
46     for (int j = 0; j < n; ++j) {
47         ans += dp[n][j];
48         ans %= MOD;
49     }
50     cout << ans << "\n";
51 }

```

3.14 Knasack2

```

1  // 01 背包，背包承重大 ( $1e9$ )，物品價值和較小
2  ( $1e5$ )
3  const int maxn = 101;
4  const int maxv = 100001;
5  int weight[maxn];
6  int cost[maxn];
7  int dp[maxv];
8
9  void solve() {
10     int n, w;
11     cin >> n >> w;
12
13     for (int i = 0; i < n; ++i) {
14         cin >> weight[i] >> cost[i];
15     }
16     fill(dp, dp + maxv, 1e18);
17
18     dp[0] = 0;
19     for (int i = 0; i < n; ++i) {
20         for (int j = maxv - 1; j >= 0; --j) {
21             if (dp[j] + weight[i] <= w) {
22                 dp[j + cost[i]] = min(dp[j] + cost[i], dp[j] + weight[i]);
23             }
24         }
25     }
26
27     for (int i = maxv - 1; i >= 0; --i) {
28         if (dp[i] != 1e18) {
29             cout << i << "\n";
30         }
31     }
32 }

```

```

30     return;
31     }
32     }
33 }
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

3.15 flowers

```

1  /*
2  LIS DP + Segment Tree -  $O(N \log N)$ 
3  有  $N$  朵花排成一列。對於每個  $i$  ( $1 \leq i \leq N$ )，
4  第  $i$  朵花的高度與美麗分別為  $h_i$  與  $a_i$ 。
5  此處  $h_1, h_2, \dots, h_N$  兩兩互異。
6
7  A 會拔掉一些花，使得剩下的花從左到右的高度為
8  單調遞增（嚴格遞增）。
9
10 求剩下花的美麗值總和的最大可能值。
11
12令  $dp[i]$  表示以第  $i$  朵花為結尾的遞增子序列所
13能取得的最大美麗值。
14
15*/
16 void solve() {
17     int n;
18     cin >> n;
19     SGT<int, MergeMax> tree(n + 1, 0);
20     vector<int> h(n), b(n);
21
22     for (int i = 0; i < n; ++i) {
23         cin >> h[i];
24     }
25     for (int i = 0; i < n; ++i) {
26         cin >> b[i];
27     }
28
29     for (int i = 0; i < n; ++i) {
30         int mx = tree.query(0, h[i]);
31         tree.modify(h[i], mx + b[i]);
32     }
33
34     cout << tree.query(0, n + 1) << "\n";
35 }

```

3.16 independent set

```

1  /*
2  DP on Trees -  $O(N)$ 
3  有一棵含  $N$  個頂點的樹，頂點編號為  $1, 2, \dots, N$ 。
4  對於每個  $i$  ( $1 \leq i \leq N-1$ )，第  $i$  條邊連接
5  頂點  $x_i$  和  $y_i$ 。
6
7  A 決定將每個頂點塗成白色或黑色，但不允許兩個
8  相鄰的頂點同時為黑色。
9
10 求將頂點塗色的方案數，對  $10^9+7$  取模。
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

9  設  $dp[i][j]$  表示以節點  $i$  為根的子樹中，在節
10 點  $i$  顏色為  $j$  時的塗色方案數（例如  $j=0$ 
11 表示白色， $j=1$  表示黑色）。
12
13 */
14 const int maxn = 100001;
15 vector<int> adj[maxn];
16 int f[maxn][2];
17 const int MOD = 1e9 + 7;
18
19 void dp(int u, int p) {
20     for (int v : adj[u]) {
21         if (v != p) {
22             dp(v, u);
23             f[u][0] = (f[v][0] + f[v][1]) % MOD;
24             f[u][1] = f[v][0] * f[u][1] % MOD;
25         }
26     }
27
28     void solve() {
29         int n;
30         cin >> n;
31
32         for (int i = 0; i < n; ++i) {
33             f[i][0] = f[i][1] = 1;
34         }
35
36         for (int i = 0; i < n - 1; ++i) {
37             int u, v;
38             cin >> u >> v;
39             u--, v--;
40             adj[u].pb(v);
41             adj[v].pb(u);
42         }
43
44         dp(0, -1);
45
46         cout << (f[0][0] + f[0][1]) % MOD << "\n";
47     }
48 }

```

3.17 counting tower

```

1  /*
2  狀態機 DP -  $O(N)$ 
3  你的任務是建造一座寬度為 2、高度為  $n$  的塔。
4  你有無限數量寬度與高度為整數的方塊。
5
6   $dp[i][0]$  = 高度為  $i$  的塔中，頂層為一個寬度為
7  2 的方塊（即該層由跨越兩欄的單一方塊覆
8  蓋）的塔的数量。
9   $dp[i][1]$  = 高度為  $i$  的塔中，頂層在該層有兩個
10 寬度為 1 的方塊（每欄各一個）的塔的数量。
11
12 */
13 void solve() {
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

10 long long n;
11 cin >> n;
12 dp[1][0] = 1;
13 dp[1][1] = 1;
14 for(int i = 2; i <= n; i++){
15     dp[i][0] = (4 * dp[i-1][0] + dp[i-1][1]) % mod;
16     dp[i][1] = (dp[i-1][0] + 2 * dp[i-1][1]) % mod;
17 }
18 cout << (dp[n][0] + dp[n][1]) % mod << endl;
19 }

```

3.18 counting numbers

```

1 /*
2 區間 DP - O(digits*10)
3 您的任務是計算在區間 a 到 b 之間，沒有任何相鄰兩位數字相同的整數的個數。
4
5 定義 dp[pos][prev_digit][is_tight][is_started] 為從位置 pos 到結尾的有效整數數量。
6 其中 prev_digit 是位置 pos-1 的數字。
7 is_tight 表示是否受原數字前綴的限制。
8 is_started 表示是否已開始構成有效數字（用以避免計入前導零）。
9 */
10
11 // dp[pos][prev_digit][is_tight][is_started]
12 int dp[20][10][2][2];
13 string s;
14
15 int f(int pos, int prev_digit, bool is_tight, bool is_started) {
16     if (pos == (int)s.size()) {
17         return 1;
18     }
19     if (dp[pos][prev_digit][is_tight][is_started] != -1) {
20         return dp[pos][prev_digit][is_tight][is_started];
21     }
22
23     int ans = 0;
24     int max_d = is_tight ? (s[pos] - '0') : 9;
25     for (int d = 0; d <= max_d; ++d) {
26         if (is_started && d == prev_digit) {
27             continue;
28         }
29
30         bool new_is_started = is_started || (d > 0);
31         bool new_is_tight = is_tight && (d == max_d);
32         ans += f(pos + 1, d, new_is_tight, new_is_started);
33     }
34 }

```

```

35 return dp[pos][prev_digit][is_tight][is_started] = ans;
36 }
37
38 int count(int x) {
39     if (x < 0) return 0;
40     s = to_string(x);
41     memset(dp, -1, sizeof(dp));
42     return f(0, 0, true, false);
43 }
44
45 void solve() {
46     int a, b;
47     cin >> a >> b;
48
49     int ca = count(a - 1);
50     int cb = count(b);
51     cout << cb - ca << "\n";
52 }

```

4 Data Structure

4.1 undo disjoint set

```

1 struct DisjointSet {
2     // save() is like recursive
3     // undo() is like return
4     int n, fa[MXN], sz[MXN];
5     vector<pair<int*, int*>> h;
6     vector<int> sp;
7     void init(int tn) {
8         n = tn;
9         for (int i = 0; i < n; i++) sz[fa[i] = i] = 1;
10        sp.clear(); h.clear();
11    }
12    void assign(int *k, int v) {
13        h.PB({k, *k});
14        *k = v;
15    }
16    void save() { sp.PB(SZ(h)); }
17    void undo() {
18        assert(!sp.empty());
19        int last = sp.back(); sp.pop_back();
20        while (SZ(h) != last) {
21            auto x = h.back(); h.pop_back();
22            *x.F = x.S;
23        }
24    }
25    int f(int x) {
26        while (fa[x] != x) x = fa[x];
27        return x;
28    }
29    void uni(int x, int y) {
30        x = f(x); y = f(y);
31        if (x == y) return;
32        if (sz[x] < sz[y]) swap(x, y);
33        assign(&sz[x], sz[x] + sz[y]);
34        assign(&fa[y], x);
35    }
36 } djs;

```

4.2 lazy propagation

```

1 // Lazy Propagation Segment Tree supporting
2 // range add, range set, and range sum queries
3
4 int n;
5 int sz;
6 struct Node {
7     int sum = 0;
8     int lazy_add = 0;
9     int lazy_set = 0;
10    bool has_set = false;
11 };
12 vector<Node> tree;
13
14 void build(const vector<int>& a) {
15     tree.resize(sz << 1);
16     for (int i = 0; i < sz; ++i) {
17         tree[i + sz].sum = i < n ? a[i] : 0;
18     }
19     for (int i = sz - 1; i >= 1; --i) {
20         tree[i].sum = tree[i << 1].sum + tree[i << 1 | 1].sum;
21     }
22 }
23
24 void apply_set(int idx, int l, int r, int val) {
25     tree[idx].sum = (r - l + 1) * val;
26     tree[idx].lazy_set = val;
27     tree[idx].lazy_add = 0;
28     tree[idx].has_set = true;
29 }
30
31 void apply_add(int idx, int l, int r, int val) {
32     if (tree[idx].has_set) {
33         tree[idx].lazy_set += val;
34     }
35     else {
36         tree[idx].lazy_add += val;
37     }
38     tree[idx].sum += (r - l + 1) * val;
39 }
40
41 void push(int idx, int l, int r) {
42     if (l == r) return;
43
44     int mid = l + (r - l) / 2;
45     int left = idx << 1, right = idx << 1 | 1;
46
47     if (tree[idx].has_set) {
48         // propagate the set operation
49         apply_set(left, l, mid, tree[idx].lazy_set);
50         apply_set(right, mid + 1, r, tree[idx].lazy_set);
51         tree[idx].has_set = false;
52         tree[idx].lazy_set = 0;
53         tree[idx].lazy_add = 0;
54     }
55     else if (tree[idx].lazy_add != 0) {
56         // propagate the add operation
57         apply_add(left, l, mid, tree[idx].lazy_add);
58         apply_add(right, mid + 1, r, tree[idx].lazy_add);
59         tree[idx].lazy_add = 0;
60     }
61 }
62
63 void range_set(int idx, int l, int r, int ql, int qr, int val) {
64     if (qr < l || ql > r) return;
65     if (ql <= l && r <= qr) {
66         apply_set(idx, l, r, val);
67         return;
68     }
69     push(idx, l, r);
70     int mid = (l + r) / 2;
71     range_set(idx << 1, l, mid, ql, qr, val);
72     range_set(idx << 1 | 1, mid + 1, r, ql, qr, val);
73     tree[idx].sum = tree[idx << 1].sum + tree[idx << 1 | 1].sum;
74 }
75
76 void range_add(int idx, int l, int r, int ql, int qr, int val) {
77     if (qr < l || ql > r) return;
78     if (ql <= l && r <= qr) {
79         apply_add(idx, l, r, val);
80         return;
81     }
82     push(idx, l, r);
83     int mid = (l + r) / 2;
84     range_add(idx << 1, l, mid, ql, qr, val);
85     range_add(idx << 1 | 1, mid + 1, r, ql, qr, val);
86     tree[idx].sum = tree[idx << 1].sum + tree[idx << 1 | 1].sum;
87 }
88
89 int range_sum(int idx, int l, int r, int ql, int qr) {
90     if (qr < l || ql > r) return 0;
91     if (ql <= l && r <= qr) return tree[idx].sum;
92
93     push(idx, l, r);
94     int mid = (l + r) / 2;
95     return range_sum(idx << 1, l, mid, ql, qr) + range_sum(idx << 1 | 1, mid + 1, r, ql, qr);
96 }
97
98 void solve() {
99     int q;
100    cin >> n >> q;
101    vector<int> a(n);
102
103    sz = 1;
104    while (sz < n) sz <= 1;
105    for (int i = 0; i < n; ++i) {
106

```

```

107        apply_add(left, l, mid, tree[idx].lazy_add);
108        apply_add(right, mid + 1, r, tree[idx].lazy_add);
109        tree[idx].lazy_add = 0;
110    }
111 }
112
113 void range_set(int idx, int l, int r, int ql, int qr, int val) {
114     if (qr < l || ql > r) return;
115     if (ql <= l && r <= qr) {
116         apply_set(idx, l, r, val);
117         return;
118     }
119     push(idx, l, r);
120     int mid = (l + r) / 2;
121     range_set(idx << 1, l, mid, ql, qr, val);
122     range_set(idx << 1 | 1, mid + 1, r, ql, qr, val);
123     tree[idx].sum = tree[idx << 1].sum + tree[idx << 1 | 1].sum;
124 }
125
126 void range_add(int idx, int l, int r, int ql, int qr, int val) {
127     if (qr < l || ql > r) return;
128     if (ql <= l && r <= qr) {
129         apply_add(idx, l, r, val);
130         return;
131     }
132     push(idx, l, r);
133     int mid = (l + r) / 2;
134     range_add(idx << 1, l, mid, ql, qr, val);
135     range_add(idx << 1 | 1, mid + 1, r, ql, qr, val);
136     tree[idx].sum = tree[idx << 1].sum + tree[idx << 1 | 1].sum;
137 }
138
139 int range_sum(int idx, int l, int r, int ql, int qr) {
140     if (qr < l || ql > r) return 0;
141     if (ql <= l && r <= qr) return tree[idx].sum;
142
143     push(idx, l, r);
144     int mid = (l + r) / 2;
145     return range_sum(idx << 1, l, mid, ql, qr) + range_sum(idx << 1 | 1, mid + 1, r, ql, qr);
146 }
147
148 void solve() {
149     int q;
150     cin >> n >> q;
151     vector<int> a(n);
152
153     sz = 1;
154     while (sz < n) sz <= 1;
155     for (int i = 0; i < n; ++i) {

```



```

108     cin >> a[i];
109 }
110
111 build(a);
112
113 while (q--) {
114     int mode, l, r, x;
115     cin >> mode >> l >> r;
116     l--, r--;
117     if (mode == 1) {
118         cin >> x;
119         range_add(1, 0, sz - 1, l, r, x)
120     }
121     else if (mode == 2) {
122         cin >> x;
123         range_set(1, 0, sz - 1, l, r, x)
124     }
125     else {
126         cout << range_sum(1, 0, sz - 1,
127                             l, r) << "\n";
128     }
129 }

```

4.3 Lazy heap

```

1 template<typename T, typename Compare = less
2 <T>>
3 struct lp {
4     priority_queue<T, vector<T>, Compare> pq;
5     unordered_map<T, int> cnt;
6     size_t sz = 0;
7     void real_remove() {
8         while (!pq.empty() && cnt[pq.top()] >
9                 0) {
10             cnt[pq.top()]--;
11             pq.pop();
12         }
13     }
14     size_t size() {
15         return sz;
16     }
17     void remove(T n) {
18         cnt[n]++;
19         sz--;
20     }
21     T top() {
22         real_remove();
23         return pq.top();
24     }
25     void push(T n) {
26         pq.push(n);
27         sz++;
28     }
29     void pop() {
30         real_remove();
31         sz--;
32         pq.pop();
33     }
34 };

```

4.4 Fenwick tree (BIT)

```

1 struct Fenwick {
2     int n;
3     vector<int> bit, prefix;
4     Fenwick(int _n) : n(_n), bit(n + 1, 0),
5         prefix(n + 1, 0) {}
6     void update(int i, int delta) {
7         for (; i <= n; i += i & -i) {
8             bit[i] += delta;
9         }
10    }
11    int query(int i) {
12        int ans = 0;
13        for (; i >= 1; i -= i & -i) {
14            ans += bit[i];
15        }
16        return ans;
17    }
18    int range(int l, int r) {
19        return query(r) - query(l - 1);
20    }
21 };
22
23 int main() {
24     int n = 5;
25     vector<int> a = {0, 1, 2, 3, 4, 5}; //
26     // 1-indexed
27     Fenwick ft(n);
28     for (int i = 1; i <= n; ++i) ft.update(i,
29         a[i]);
30
31     cout << ft.query(3) << "\n"; // 6
32     cout << ft.range(2, 4) << "\n"; // 9
33
34     ft.update(3, 10); //
35     a[3] += 10
36     cout << ft.query(3) << "\n"; // 16
37     cout << ft.range(1, 5) << "\n"; // 25
38 }

```

4.5 BIT - range update + range prefix sum

```

1 int n, q;
2
3 void solve() {
4     cin >> n >> q;
5     Fenwick bit_values(n), bit_count(n);
6
7     while (q--) {
8         int mode, l, r, val;
9         cin >> mode;
10        if (mode == 0) {
11            cin >> l >> r >> val;
12            bit_values.update(l, val);
13            bit_count.update(l, val * (l -
14                1));
15            bit_values.update(r + 1, -val);
16            bit_count.update(r + 1, -val * r
17                );
18        }
19    }
20 }

```

```

14 }
15 else {
16     cin >> l >> r;
17     int pref_l = bit_values.query(l
18         - 1) * (l - 1) - bit_count.
19         query(l - 1);
20     int pref_r = bit_values.query(r)
21         * r - bit_count.query(r);
22     cout << pref_r - pref_l << "\n";
23 }
24 }

```

4.6 Order Statistics Tree

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4
5 template <class T>
6 using ordered_set =
7     tree<T, null_type, less<T>, rb_tree_tag,
8         tree_order_statistics_node_update>;
9
10 int main() {
11     int n;
12     cin >> n;
13     ordered_set<int> st;
14     vector<int> p(n);
15     for (int i = 0; i < n; ++i) {
16         st.insert(i);
17         cin >> p[i];
18     }
19     for (int i = 0; i < n; ++i) {
20         int ind;
21         cin >> ind;
22         ind--;
23         int pos = *st.find_by_order(ind);
24         cout << st.order_of_key(pos) << ' ';
25         st.erase(pos);
26         cout << p[pos] << (i == n - 1 ? '\n' : '
27         ');
28     }
29 }

```

4.7 Trie (Prefix tree)

```

1 struct trie {
2     int n = 0;
3     trie *a[2];
4     trie() {
5         a[0] = a[1] = nullptr;
6     }
7
8     void insert(int k) {
9         trie *curr = this;
10        for (int i = 63; i >= 0; --i) {
11            bool bit = (k & (1LL << i)) > 0;
12            if (curr->a[bit] == nullptr) {
13                curr->a[bit] = new trie();
14            }
15        }
16    }
17 }

```

```

14 }
15 curr = curr->a[bit];
16 curr->n++;
17 }
18
19 void erase(int k) {
20     trie *curr = this;
21     for (int i = 63; i >= 0; --i) {
22         bool bit = (k & (1LL << i)) > 0;
23         curr = curr->a[bit];
24         curr->n--;
25     }
26 }
27
28 int query(int k) {
29     trie *curr = this;
30     int x = 0;
31     for (int i = 63; i >= 0; --i) {
32         x ^= (1LL << i) & k;
33         x ^= 1LL << i;
34         bool bit = (k & (1LL << i)) ==
35             0;
36         if (curr->a[bit] == nullptr ||
37             curr->a[bit]->n == 0)
38             x ^= 1LL << i, bit ^= 1;
39         curr = curr->a[bit];
40     }
41     return x;
42 }

```

4.8 segment tree

```

1 // Segment tree
2 template<typename value_t, class merge_t>
3 class SGT {
4     int n;
5     vector<value_t> t;
6     value_t defa;
7     merge_t merge;
8 public:
9     explicit SGT(int _n, value_t _defa,
10         const merge_t& _merge = merge_t{})
11         : n(_n), t(2 * n), defa(_defa),
12             merge(_merge) {}
13
14     void modify(int p, const value_t& x) {
15         for (t[p += n] = x; p > 1; p >>= 1)
16             t[p >> 1] = merge(t[p], t[p ^
17                 1]);
18     }
19
20     value_t query(int l, int r) { return
21         query(l, r, defa); }
22
23     value_t query(int l, int r, value_t init)
24     {
25         for (l += n, r += n; l < r; l >>= 1,
26             r >>= 1) {
27             if (l & 1) init = merge(init, t[
28                 l++]);
29         }
30     }
31 }

```

```

22     if (r & 1) init = merge(init, t
23         [--r]);
24     }
25     return init;
26 }
27 };
28 // Custom merge for range minimum + index
29 struct MergeMin {
30     pair<int, int> operator()(const pair<int
31         , int>& a,
32         const pair<int
33             , int>& b)
34         const {
35             if (a.first != b.first) return (a.
36                 first < b.first) ? a : b;
37             return (a.second < b.second) ? a : b
38                 ; // tie-break on index
39         }
40 };
41 int main() {
42     int n = 6;
43     SGT<pair<int, int>, MergeMin> tree(n, {
44         INT_MAX, -1});
45
46     vector<int> a = {5, 3, 6, 1, 4, 2};
47     for (int i = 0; i < n; ++i)
48         tree.modify(i, {a[i], i});
49
50     auto [min_val, min_idx] = tree.query(1,
51         5); // range [1, 5)
52     cout << "Min value in [1, 5): " <<
53         min_val << ", at index " << min_idx
54         << "\n";
55
56     return 0;
57 }

```

4.9 BIT range update point query

```

1 // Fenwick Tree (Binary Indexed Tree) for
2 // Range Updates and Point Queries
3
4 template<typename T>
5 class BIT {
6     #define ALL(x) begin(x), end(x)
7     private:
8         vector<T> arr;
9         int n;
10        inline int lowbit(int x) { return x & (-
11            x); }
12        void addInternal(int s, T v) {
13            while (s > 0) {
14                arr[s] += v;
15                s -= lowbit(s);
16            }
17        }
18    public:
19        void init(int n_) {
20            n = n_;
21            arr.resize(n + 1);
22            fill(ALL(arr), 0);

```

```

21     }
22     void add(int l, int r, T v) {
23         // add v to interval [l, r], 1-based
24         addInternal(l, -v);
25         addInternal(r, v);
26     }
27     T query(int x) {
28         // value at index x
29         T res = 0;
30         while (x <= n) {
31             res += arr[x];
32             x += lowbit(x);
33         }
34         return res;
35     }
36     #undef ALL
37 };
38
39 int main() {
40     BIT<int> bit;
41     bit.init(5);
42
43     // add +3 to indices 1..3
44     bit.add(0, 3, 3);
45
46     // add +2 to indices 3..5
47     bit.add(2, 5, 2);
48
49     cout << "Value at 1 = " << bit.query(1)
50         << "\n"; // expect 3
51     cout << "Value at 3 = " << bit.query(3)
52         << "\n"; // expect 3+2=5
53     cout << "Value at 5 = " << bit.query(5)
54         << "\n"; // expect 2
55 }

```

4.10 DSU

```

1 // fast disjoint set union
2 class DSU {
3     vector<int> pa, sz;
4     public:
5         explicit DSU(int n) : pa(n, -1), sz(n,
6             1) {}
7         int find(int x) { // collapsing find
8             return pa[x] == -1 ? x : pa[x] =
9                 find(pa[x]);
10        }
11        void unite(int x, int y) { // weighted
12            union
13            auto rx{find(x)}, ry{find(y)};
14            if (rx == ry) return;
15            if (sz[rx] < sz[ry]) swap(rx, ry);
16            pa[ry] = rx, sz[rx] += sz[ry];
17        }
18 };

```

5 Geometry

5.1 cross. product

```

1 // If cross(a,b,c) > 0, c is to the left of
2 // line ab
3 // If cross(a,b,c) < 0, c is to the right of
4 // line ab
5 // If cross(a,b,c) = 0, a, b, c are
6 // collinear
7
8 // point struct version
9 // 2D Point structure
10 struct Point {
11     double x, y;
12 };
13
14 // Cross product (scalar in 2D)
15 double cross(const Point& a, const Point& b,
16     const Point& c) {
17     // Computes (b - a) x (c - a)
18     return (b.x - a.x) * (c.y - a.y) -
19         (b.y - a.y) * (c.x - a.x);
20 }
21
22 // std::complex version
23 typedef std::complex<double> point;
24 #define x real()
25 #define y imag()
26
27 double cross(const point &a, const point &b)
28 {
29     return (std::conj(a) * b).imag();
30 }

```

5.2 Check Point in Polygon

```

1 /*
2 We can cast a ray from the point P going in
3 any fixed direction (people usually go
4 to the right).
5 If the point is located on the outside of
6 the polygon the ray will intersect its
7 edges an even number of times.
8 If the point is on the inside of the polygon
9 then it will intersect the edge an odd
10 number of times.
11 */
12
13 struct Point {
14     int x, y;
15 };
16
17 int cross(const Point& a, const Point& b,
18     const Point& c) {
19     // return (c - a) x (b - a)
20     int ans = (c.x - a.x) * (b.y - a.y) -
21         (c.y - a.y) * (b.x - a.x);
22     if (ans == 0) return 0;
23     return ans > 0 ? 1 : -1;

```

```

16 }
17
18 bool intersect(const Point& a, const Point&
19     b, const Point& c, const Point& d) {
20     int c1 = cross(a, b, c);
21     int c2 = cross(a, b, d);
22     int c3 = cross(c, d, a);
23     int c4 = cross(c, d, b);
24
25     return (c1 * c2 < 0 && c3 * c4 < 0);
26 }
27
28 bool onSegment(const Point& a, const Point&
29     b, const Point& c) {
30     return (cross(a, b, c) == 0 &&
31         min(a.x, b.x) <= c.x && c.x <=
32             max(a.x, b.x) &&
33             min(a.y, b.y) <= c.y && c.y <=
34                 max(a.y, b.y));
35 }
36
37 void solve() {
38     int n, m;
39     cin >> n >> m;
40     vector<Point> poly(n);
41
42     for (int i = 0; i < n; ++i) {
43         int a, b;
44         cin >> a >> b;
45         poly[i] = {a, b};
46     }
47
48     // set a point at outside point to form
49     // the ray
50     Point outside = {(int)2e9 + 7, (int)2e9
51         + 13};
52     for (int i = 0; i < m; ++i) {
53         int a, b;
54         bool found = false;
55         cin >> a >> b;
56         Point p = {a, b};
57
58         int cnt = 0;
59         if (intersect(poly.back(), poly[0],
60             p, outside)) {
61             cnt++;
62         }
63         if (onSegment(poly.back(), poly[0],
64             p)) {
65             cout << "BOUNDARY\n";
66             found = true;
67         }
68         for (int j = 0; j < n - 1 && !found;
69             ++j) {
70             if (intersect(poly[j], poly[j +
71                 1], p, outside)) {
72                 cnt++;
73             }
74             if (onSegment(poly[j], poly[j +
75                 1], p)) {
76                 cout << "BOUNDARY\n";
77                 found = true;
78             }
79         }
80         if (found) continue;
81         if (cnt & 1) {

```

```

71     cout << "INSIDE\n";
72 }
73 else {
74     cout << "OUTSIDE\n";
75 }
76 }
77 }

```

5.3 Point

```

1  /**
2   * Author: Ulf Lundstrom
3   * Date: 2009-02-26
4   * License: CC0
5   * Source: My head with inspiration from
6   *         tinyKACTL
7   * Description: Class to handle points in
8   *             the plane.
9   * T can be e.g. double or long long. (
10  * Avoid int.)
11  * Status: Works fine, used a lot
12  */
13 #pragma once
14
15 template <class T> int sgn(T x) { return (x
16 > 0) - (x < 0); }
17
18 template <class T>
19 struct Point {
20     typedef Point P;
21     T x, y;
22     explicit Point(T x=0, T y=0) : x(x), y(y)
23     {}
24     bool operator<(P p) const { return tie(x,y)
25 < tie(p.x,p.y); }
26     bool operator==(P p) const { return tie(x,
27 y)==tie(p.x,p.y); }
28     P operator+(P p) const { return P(x+p.x, y
29 +p.y); }
30     P operator-(P p) const { return P(x-p.x, y
31 -p.y); }
32     P operator*(T d) const { return P(x*d, y*d
33 ); }
34     P operator/(T d) const { return P(x/d, y/d
35 ); }
36     T dot(P p) const { return x*p.x + y*p.y; }
37     T cross(P p) const { return x*p.y - y*p.x;
38 }
39     T cross(P a, P b) const { return (a-*this)
40 .cross(b-*this); }
41     T dist2() const { return x*x + y*y; }
42     double dist() const { return sqrt((double)
43 dist2()); }
44     // angle to x-axis in interval [-pi, pi]
45     double angle() const { return atan2(y, x);
46 }
47     P unit() const { return *this/dist(); } //
48     makes dist()=1
49     P perp() const { return P(-y, x); } //
50     rotates +90 degrees
51     P normal() const { return perp().unit(); }
52     // returns point rotated 'a' radians ccw
53     around the origin
54     P rotate(double a) const {

```

```

36     return P(x*cos(a)-y*sin(a),x*sin(a)+y*
37 cos(a)); }
38 friend ostream& operator<<(ostream& os, P
39 p) {
40     return os << "(" << p.x << ", " << p.y <<
41     ")"; }
42 };

```

5.4 Polygon Lattice Points

```

1  /**
2   * Pick's Theorem:
3   * For a simple polygon with integer
4   * coordinates, the area A, the number of
5   * interior lattice points a
6   * and the number of boundary lattice points b
7   * satisfy:
8   * A = a + b/2 - 1
9   */
10 #pragma once
11
12 int countOnSegment(const Point& a, const
13 Point& b) {
14     int dx = abs(a.x - b.x);
15     int dy = abs(a.y - b.y);
16     return __gcd(dx, dy);
17 }
18
19 void solve() {
20     int n;
21     cin >> n;
22     vector<Point> poly(n);
23
24     for (int i = 0; i < n; ++i) {
25         cin >> poly[i].x >> poly[i].y;
26     }
27
28     // b: boundary points
29     int b = countOnSegment(poly.back(), poly
30 [0]);
31     for (int i = 0; i < n - 1; ++i) {
32         b += countOnSegment(poly[i], poly[i
33 + 1]);
34     }
35
36     int area2 = abs(polygonArea2(poly));
37     // a: interior points
38     int a = (area2 + 2 - b) / 2;
39     cout << a << " " << b << "\n";
40 }

```

5.5 line intersect

```

1  // 2D Point structure
2  struct Point {
3      double x, y;

```

```

4  };
5
6  // Cross product (scalar in 2D)
7  double cross(const Point& a, const Point& b,
8  const Point& c) {
9      // Computes (b - a) x (c - a)
10     return (b.x - a.x) * (c.y - a.y) -
11     (b.y - a.y) * (c.x - a.x);
12 }
13
14 // Check if value is between two others (
15 // with endpoints allowed)
16 bool between(double a, double b, double x) {
17     return min(a, b) <= x && x <= max(a, b);
18 }
19
20 // Check if point c lies on segment ab
21 bool onSegment(const Point& a, const Point&
22 b, const Point& c) {
23     return cross(a, b, c) == 0 &&
24     between(a.x, b.x, c.x) &&
25     between(a.y, b.y, c.y);
26 }
27
28 // Main intersection check
29 bool segmentsIntersect(const Point& A, const
30 Point& B, const Point& C, const
31 Point& D) {
32     double c1 = cross(A, B, C);
33     double c2 = cross(A, B, D);
34     double c3 = cross(C, D, A);
35     double c4 = cross(C, D, B);
36
37     // General case
38     if (c1 * c2 < 0 && c3 * c4 < 0) return
39     true;
40
41     // Special cases: collinear +
42     // overlapping
43     if (c1 == 0 && onSegment(A, B, C))
44         return true;
45     if (c2 == 0 && onSegment(A, B, D))
46         return true;
47     if (c3 == 0 && onSegment(C, D, A))
48         return true;
49     if (c4 == 0 && onSegment(C, D, B))
50         return true;
51
52     return false;
53 }
54
55 int main() {
56     Point A{0, 0}, B{4, 4};
57     Point C{0, 4}, D{4, 0};
58
59     if (segmentsIntersect(A, B, C, D))
60         cout << "Segments intersect\n";
61     else
62         cout << "Segments do not intersect\n";
63
64     return 0;
65 }

```

5.6 Polygon area

```

1  /**
2   * Author: Ulf Lundstrom
3   * Date: 2009-03-21
4   * License: CC0
5   * Source: tinyKACTL
6   * Description: Returns twice the signed
7   *             area of a polygon.
8   * Clockwise enumeration gives negative
9   * area. Watch out for overflow if using
10  * int as T!
11  * Status: Stress-tested and tested on
12  * kattis:polygonarea
13  */
14 #pragma once
15
16 #include "Point.h"
17
18 template <class T>
19 T polygonArea2(vector<Point<T>>& v) {
20     T a = v.back().cross(v[0]);
21     for (int i = 0; i < (int)v.size() - 1;
22         ++i) {
23         a += v[i].cross(v[i+1]);
24     }
25     return a;
26 }
27
28 int main() {
29     // Example: square with vertices (0,0),
30     // (0,1), (1,1), (1,0)
31     vector<Point<long long>> poly = {
32     {0,0}, {0,1}, {1,1}, {1,0}
33     };
34
35     long long area2 = polygonArea2(poly);
36     cout << "Twice signed area = " << area2
37     << "\n";
38     cout << "Absolute area = " << abs(area2)
39     / 2.0 << "\n";
40
41     return 0;
42 }

```

5.7 using std::complex

```

1  #include <iostream>
2  #include <complex>
3  #include <cmath>
4
5  typedef std::complex<double> point;
6  #define x real()
7  #define y imag()
8
9  constexpr double PI = std::acos(-1.0);
10 // conj(a) = a 的共軛 · reflection of a
11 // across x-axis
12
13 // Basic operations
14 double dot(const point &a, const point &b) {
15     return (std::conj(a) * b).real(); }

```



```

14 double cross(const point &a, const point &b)
15 { return (std::conj(a) * b).imag(); }
16 // Projections / reflections / geometry
17 // helpers
18 point project_onto_vector(const point &p,
19 const point &v) {
20 return v * (dot(p, v) / std::norm(v));
21 }
22 point project_onto_line(const point &p,
23 const point &a, const point &b) {
24 return a + (b - a) * (dot(p - a, b - a)
25 / std::norm(b - a));
26 }
27 point reflect_across_line(const point &p,
28 const point &a, const point &b) {
29 return a + std::conj((p - a) / (b - a))
30 * (b - a);
31 }
32 point intersection(const point &a, const
33 point &b, const point &p, const point &q
34 ) {
35 double c1 = cross(p - a, b - a), c2 =
36 cross(q - a, b - a);
37 return (c1 * q - c2 * p) / (c1 - c2); //
38 // undefined if parallel (divide by
39 // zero)
40 }
41 double angle_between(const point &a, const
42 point &b) {
43 // angle of vector b - a
44 return std::arg(b - a);
45 }
46 double angle_ABC(const point &a, const point
47 &b, const point &c) {
48 double r = std::remainder(std::arg(a - b)
49 - std::arg(c - b), 2.0 * PI);
50 return std::abs(r);
51 }
52 int main() {
53 // sample
54 point a = 2.0; // (2,0)
55 point b(3.0, 7.0); // (3,7)
56 std::cout << a << ' ' << b << '\n'; //
57 (2,0) (3,7)
58 std::cout << a + b << '\n'; //
59 (5,7)
60 // usage examples
61 point p1(3, 2), p2(2, -7);
62 std::cout << "p1 + p2 = " << p1 + p2 <<
63 '\n'; // (5, -5)
64 std::cout << "p1 - p2 = " << p1 - p2 <<
65 '\n'; // (1, 9)
66 std::cout << "3.0 * p1 = " << 3.0 * p1
67 << '\n'; // (9, 6)
68 std::cout << "p1 / 5.0 = " << p1 / 5.0
69 << '\n'; // (0.6, 0.4)
70 // dot / cross via complex

```

```

59 std::cout << "dot(p1,p2) = " << dot(p1,
60 p2) << '\n';
61 std::cout << "cross(p1,p2) = " << cross(
62 p1, p2) << '\n';
63 // distances, angle, rotation, polar/
64 // cartesian
65 std::cout << "squared dist = " << std::
66 norm(p1 - p2) << '\n';
67 std::cout << "euclid dist = " << std::
68 abs(p1 - p2) << '\n';
69 std::cout << "angle p1->p2 = " <<
70 angle_between(p1, p2) << '\n';
71 std::cout << "angle ABC = " << angle_ABC
72 (point(1,0), point(0,0), point(0,1))
73 << '\n';
74 // project / reflect / intersect
75 // examples
76 point v(1, 1);
77 point proj = project_onto_vector(p1, v);
78 std::cout << "project p1 onto v = " <<
79 proj << '\n';
80 point lineA(0,0), lineB(2,0);
81 std::cout << "project p1 onto line = "
82 << project_onto_line(p1, lineA,
83 lineB) << '\n';
84 std::cout << "reflect p1 across line = "
85 << reflect_across_line(p1, lineA,
86 lineB) << '\n';
87 // intersection example
88 point r1a(0,0), r1b(1,1);
89 point r2a(0,1), r2b(1,0);
90 std::cout << "intersection = " <<
91 intersection(r1a, r1b, r2a, r2b) <<
92 '\n';
93 // polar/cartesian and rotation
94 point polar_pt = std::polar(5.0, PI/4);
95 std::cout << "polar(5,PI/4) = " <<
96 polar_pt << '\n';
97 point rotated = p1 * std::polar(1.0, PI
98 /2); // rotate p1 by 90 degrees
99 // about origin
100 std::cout << "rotate p1 by 90deg = " <<
101 rotated << '\n';
102 return 0;

```

5.8 point distance from a line

```

1 // calculate area using cross product and
2 // divide by base length
3 double point_line_distance(const point &p,
4 const point &a, const point &b) {
5 // area = 0.5 * |cross(b - a, p - a)|
6 // base = |b - a|
7 // height = 2 * area / base = |cross(b -
8 a, p - a)| / |b - a|

```

```

6 std::abs(cross(b - a, p - a)) / std::abs
7 (b - a);
8 }
9 // Prim's algorithm
10 vector<tuple<int, int, long long>> Prim(
11 const vector<vector<pair<int, long long
12 >>>& adj) {
13 const auto& n = adj.size();
14 vector<tuple<int, int, long long>> mst
15 {};
16 vector<bool> found(n, false);
17 using ti = tuple<long long, int, int>;
18 priority_queue<ti, vector<ti>, greater<
19 ti>> pq{};
20 found[0] = true;
21 for (auto& [v, w] : adj[0]) pq.emplace(w
22 , 0, v);
23 for (int i = 0; i < n - 1; ++i) {
24 int mn, u, v;
25 do {
26 tie(mn, u, v) = pq.top(), pq.pop()
27 ();
28 } while (found[v]);
29 found[v] = true, mst.emplace_back(u,
30 v, mn);
31 for (auto& [x, w] : adj[v]) pq.
32 emplace(w, v, x);
33 }
34 return mst;
35 }

```

6 Graph

6.1 Prim

6.2 Eulerian cycle

```

1 // Eulerian cycle in an undirected graph
2 vector<int> euler_cycle(vector<vector<pair<
3 int, int>>>& adj, int w = 0) {
4 int n{adj.size()}, m{};
5 for (int v{0}; v < n; ++v) m += adj[v].
6 size();
7 m /= 2;
8 vector<int> res{};
9 stack<pair<int, int>> stk{};
10 stk.emplace(w, -1);
11 vector<int> nxt(n);
12 vector<bool> usd(m);
13 while (!stk.empty()) {
14 auto [u, i]{stk.top()};

```

```

15 while (nxt[u] < adj[u].size() && usd
16 [adj[u][nxt[u]].second]) ++nxt[u];
17 if (nxt[u] < adj[u].size()) {
18 auto [v, j]{adj[u][nxt[u]]};
19 ++nxt[u], usd[j] = true, stk.
20 emplace(v, j);
21 } else {
22 if (i != -1) res.push_back(i);
23 stk.pop();
24 }
25 return res;
26 }
27 int main() {
28 int n = 4; // number of vertices
29 vector<vector<pair<int, int>>> adj(n);
30 // Add edges with edge IDs
31 int eid = 0;
32 auto add_edge = [&](int u, int v) {
33 adj[u].push_back({v, eid});
34 adj[v].push_back({u, eid});
35 ++eid;
36 };
37 add_edge(0, 1);
38 add_edge(1, 2);
39 add_edge(2, 3);
40 add_edge(3, 0);
41 vector<int> cycle = euler_cycle(adj, 0);
42 cout << "Euler cycle (edge IDs in order)
43 : ";
44 for (int id : cycle) cout << id << " ";
45 cout << "\n";
46 return 0;
47 }

```

6.3 maximum weight bipartite matching

```

1 // maximum weight matching in a weighted
2 // bipartite graph
3 // Hungarian Algorithm - O(V^3)
4 class bipartiteGraph {
5 int n;
6 vector<long long> lx, ly;
7 vector<bool> vx, vy;
8 queue<int> qu{}; // only X's vertices
9 vector<int> py;
10 vector<long long> dy; // smallest (lx[x]
11 + ly[y] - w[x][y])
12 vector<int> pdy; // & which x
13 void relax(int x) {
14 for (int y{0}; y < n; ++y)
15 if (lx[x] + ly[y] - w[x][y] < dy
16 [y])

```

```

14         dy[y] = lx[x] + ly[y] - w[x
15             ][y], pdy[y] = x;
16     }
17     void augment(int y) {
18         while (y != -1) {
19             int x{qu.front()}; qu.pop();
20             mx[x] = y, my[y] = x;
21             y = yy;
22         }
23     }
24     bool bfs() {
25         while (!qu.empty()) {
26             int x{qu.front()}; qu.pop();
27             for (int y{0}; y < n; ++y) {
28                 if (!vy[y] && lx[x] + ly[y]
29                     == w[x][y]) {
30                     vy[y] = true, py[y] = x;
31                     if (my[y] == -1) return
32                         augment(y, true);
33                     int xx{my[y]};
34                     qu.push(x), vx[xx] =
35                         true, relax(xx);
36                 }
37             }
38             return false;
39         }
40     }
41     void relabel() {
42         long long d{numeric_limits<long long>
43             >::max()};
44         for (int y{0}; y < n; ++y) if (!vy[y]
45             ) d = min(d, dy[y]);
46         for (int x{0}; x < n; ++x) if (vx[x]
47             ) lx[x] -= d;
48         for (int y{0}; y < n; ++y) if (vy[y]
49             ) ly[y] += d;
50         for (int y{0}; y < n; ++y) if (!vy[y]
51             ) dy[y] -= d;
52     }
53     bool expand() { // expand one layer with
54         new equality edges
55         for (int y{0}; y < n; ++y) {
56             if (!vy[y] && dy[y] == 0) {
57                 vy[y] = true, py[y] = pdy[y]
58                 ];
59                 if (my[y] == -1) return
60                     augment(y, true);
61                 int xx{my[y]};
62                 qu.push(xx), vx[xx] = true,
63                     relax(xx);
64             }
65         }
66         return false;
67     }
68     public:
69         vector<vector<long long>> w;
70         vector<int> mx, my;
71         bipartiteGraph(int _n) : n{_n}, lx(n),
72             ly(n), vx(n), vy(n), py(n), dy(n),
73             pdy(n),
74             w(n, vector<long long>(n, 0)), mx(n,
75                 -1), my(n, -1) {}
76         long long Hungarian() {
77             for (int i{0}; i < n; ++i) {
78                 lx[i] = ly[i] = 0;
79                 for (int j{0}; j < n; ++j)
80                     lx[i] = max(lx[i], w[i][j]);
81                 mx[i] = my[i] = -1;
82             }
83             for (int x{0}; x < n; ++x) {
84                 fill(vx.begin(), vx.end(), false
85                     );
86                 fill(vy.begin(), vy.end(), false
87                     );
88                 fill(dy.begin(), dy.end(),
89                     numeric_limits<long long>::
90                         max());
91                 qu = {}, qu.push(x), vx[x] =
92                     true, relax(x);
93                 while (!bfs()) {
94                     relabel();
95                     if (expand()) break;
96                 }
97             }
98             long long weight{0};
99             for (int x{0}; x < n; ++x) weight +=
100                 w[x][mx[x]];
101             return weight;
102         }
103     };
104     // usage example:
105     int main() {
106         int n = 4;
107         bipartiteGraph g(n);
108         long long mat[4][4] = {
109             {9, 2, 7, 8},
110             {6, 4, 3, 7},
111             {5, 8, 1, 8},
112             {7, 6, 9, 4}
113         };
114         g.Hungarian();
115         std::cout << "Maximum weight: " <<
116             g.maxWeight << "\n";
117         for (int i = 0; i < n; ++i)
118             for (int j = 0; j < n; ++j)
119                 g.w[i][j] = mat[i][j];
120         long long maxWeight = g.Hungarian();
121         std::cout << "Maximum weight: " <<
122             maxWeight << "\n";
123         for (int i = 0; i < n; ++i)
124             for (int j = 0; j < n; ++j)
125                 std::cout << "X " << i << "
126                     matched
127                     with Y " << g.mx[i] << "\n";
128     }
129 }
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

6.5 maximum cardinality bipartite matching

```

1 class BipartiteMatching {
2     vector<bool> vis;
3     vector<int> mx{}, my{};
4     bool dfs(const vector<vector<int>>& adj,
5             int x) {
6         for (int y : adj[x]) {
7             if (vis[y]) continue;
8             vis[y] = true;
9             if (my[y] == -1 || dfs(adj, my[y])) {
10                 mx[x] = y; my[y] = x;
11                 return true;
12             }
13         }
14         return false;
15     }
16 public:
17     pair<vector<int>, vector<int>> operator
18     ()(const vector<vector<int>>& adj,
19        int ny) {
20         vis.assign(ny, false);
21         mx.assign((int)adj.size(), -1);
22         my.assign(ny, -1);
23         for (int x = 0; x < (int)adj.size();
24             ++x) {
25             fill(vis.begin(), vis.end(),
26                  false);
27             dfs(adj, x);
28         }
29         return {mx, my};
30     }
31 };
32
33 // Usage example:
34 int main() {
35     // Left partition has 4 nodes (0..3),
36     // right partition has 4 nodes (0..3).
37     vector<vector<int>> adj(4);
38     adj[0] = {0, 1};
39     adj[1] = {1, 2};
40     adj[2] = {2};
41     adj[3] = {2, 3};
42
43     BipartiteMatching bm;
44     auto res = bm(adj, 4);
45     const auto& mx = res.first;
46
47     cout << "Matched pairs (Left -> right):\n";
48     for (int i = 0; i < (int)mx.size(); ++i)
49         if (mx[i] != -1)
50             cout << i << " -> " << mx[i] <<
51             "\n";
52 }

```

6.6 Floyd-Warshall

```
1 // Floyd-Warshall algorithm
```

```

1 template<typename T>
2 vector<vector<optional<T>>> Floyd_Warshall(
3     const vector<vector<optional<T>>>& adj)
4 {
5     const auto& n{adj.size()};
6     auto d{adj};
7
8     for (int i{0}; i < n; ++i) d[i][i] = 0;
9     for (int k{0}; k < n; ++k)
10         for (int i{0}; i < n; ++i)
11             for (int j{0}; j < n; ++j) {
12                 if (!d[i][k] || !d[k][j])
13                     continue; // no value
14                 if (!d[i][j] || d[i][j] > d[
15                     i][k].value() + d[k][j].
16                     value())
17                     d[i][j] = d[i][k].value
18                     () + d[k][j].value()
19                     ;
20             }
21     return d;
22 }

```

6.7 MST

```

1 // Kruskal' s algorithm
2 vector<tuple<int, int, long long>> Kruskal(
3     int n, vector<tuple<long long, int, int>
4     >>& edges) {
5     vector<tuple<int, int, long long>> mst
6     {};
7     DSU dsu{n};
8
9     sort(edges.begin(), edges.end());
10    for (auto& [w, u, v] : edges)
11        if (dsu.find(u) != dsu.find(v))
12            dsu.unionn(u, v), mst.
13            emplace_back(u, v, w);
14    return mst;
15 }

```

6.8 Hopcroft-Karp

```

1 // maximum cardinality bipartite matching in
2 // unweighted bipartite graph
3 // Hopcroft-Karp algorithm - O(EV)
4 class bipartiteGraph {
5     int nx, ny;
6     vector<int> dx, dy;
7     bool dfs(int y) {
8         for (auto& x : adjy[y]) {
9             if (dx[x] + 1 != dy[y]) continue
10             ;
11             if (mx[x] == -1 || dfs(mx[x])) {
12                 my[y] = x, mx[x] = y;
13                 dy[y] = -1; // augmented -
14                 // remove from BFS forest
15                 return true;
16             }
17         }
18     }
19 }

```

```

15 }
16 dy[y] = -1; // no augmenting path -
17 // remove from BFS forest
18 return false;
19 }
20 bool bfs() {
21     fill(dx.begin(), dx.end(), -1);
22     fill(dy.begin(), dy.end(), -1);
23     queue<int> qu1, qu2{};
24     for (int x{0}; x < nx; ++x) // use
25         all unmatched x's as roots
26         if (mx[x] == -1) qu1.push(x), dx[
27             x] = 0;
28
29     bool found{false};
30     while (!qu1.empty() && !found) { //
31         stop at the level found
32         while (!qu2.empty()) {
33             auto x{qu1.front()}; qu1.pop()
34             ;
35             for (auto& y : adjx[x])
36                 if (dy[y] == -1 && my[y]
37                     != x) {
38                     dy[y] = dx[x] + 1;
39                     if (my[y] == -1)
40                         found = true;
41                     else qu2.push(my[y])
42                         , dx[my[y]] = dy
43                         [y] + 1;
44                 }
45             qu.swap(qu2);
46         }
47         return found;
48     }
49     vector<vector<int>> adjx, adjy;
50 public:
51     vector<int> mx, my;
52     bipartiteGraph(int _nx, int _ny) : nx{
53         _nx}, ny{_ny}, dx(nx), dy(ny)
54         , adjx(nx), adjy(ny), mx(nx, -1), my
55         (ny, -1) {}
56     void addEdge(int x, int y) {
57         adjx[x].push_back(y), adjy[y].
58         push_back(x);
59     }
60     int Hopcroft_Karp() {
61         int c{0};
62         while (bfs()) {
63             for (int y{0}; y < ny; ++y)
64                 if (my[y] == -1 && dy[y] !=
65                     -1) c += dfs(y);
66         }
67         return c;
68     }
69 };
70
71 // Usage example:
72 int main() {
73     // Left partition has 4 nodes (0..3),
74     // right partition has 4 nodes (0..3).
75     bipartiteGraph bg(4, 4);
76     bg.addEdge(0, 0);
77     bg.addEdge(0, 1);
78     bg.addEdge(1, 1);
79     bg.addEdge(1, 2);

```

```

66     bg.addEdge(2, 2);
67     bg.addEdge(3, 2);
68     bg.addEdge(3, 3);
69
70     int max_matching = bg.Hopcroft_Karp();
71     cout << "Maximum matching size: " <<
72     max_matching << "\n";
73     cout << "Matched pairs (Left -> right):\n";
74     for (int i = 0; i < (int)bg.mx.size();
75         ++i)
76         if (bg.mx[i] != -1)
77             cout << i << " -> " << bg.mx[i]
78             << "\n";
79 }

```

6.9 Dijkstra

```

1 // Dijkstra algorithm
2 template<typename T>
3 vector<optional<T>> Dijkstra(const vector<
4     vector<pair<int, T>>& adj, int s) {
5     const auto& n{adj.size()};
6     vector<optional<T>> d(n, nullopt);
7     d[s] = 0;
8
9     vector<bool> found(n, false);
10    using pq_t = pair<T, int>;
11    priority_queue<pq_t, vector<pq_t>,
12        greater<pq_t>> pq{};
13    pq.emplace(0, s);
14    while (!pq.empty()) {
15        auto [_, u]{pq.top()}; pq.pop();
16        if (found[u]) continue;
17        found[u] = true;
18        for (auto& [v, w] : adj[u])
19            if (!d[v] || d[v] > d[u].value()
20                + w) {
21                d[v] = d[u].value() + w;
22                pq.emplace(d[v].value(), v);
23            }
24    }
25    return d;
26 }

```

6.10 Dinic

```

1 int n, s, t, cnt, inf = 1e17;
2 vector<vector<int>> adj;
3 int level[100000], ptr[100000], rev[100000],
4     to[100000], cap[100000];
5 void add(int a, int b, int c) {
6     int id = cnt++, rid = cnt++;
7     to[id] = b;
8     to[rid] = a;
9     cap[id] = c;
10    cap[rid] = 0;
11
12    rev[id] = rid;

```

```

12 rev[rid] = id;
13 adj[a].push_back(id);
14 adj[b].push_back(rid);
15 }
16 bool bfs() {
17     memset(level, -1, sizeof(level));
18     queue<int> q;
19     q.push(s);
20     level[s] = 0;
21
22     while(!q.empty()) {
23         int curr = q.front(); q.pop();
24         for(int id: adj[curr]) {
25             int next = to[id];
26             if(cap[id] > 0 && level[next] ==
27                 -1) {
28                 level[next] = level[curr] +
29                     1;
30                 q.push(next);
31             }
32         }
33     }
34     return level[t] != -1;
35 }
36 int dfs(int curr, int flow) {
37     if(curr == t || flow == 0) return flow;
38     for(; ptr[curr] < adj[curr].size(); ptr[
39         curr]++) {
40         int id = adj[curr][ptr[curr]];
41         int next = to[id];
42         if(cap[id] > 0 && level[next] ==
43             level[curr] + 1) {
44             int temp = dfs(next, min(flow,
45                 cap[id]));
46             if(temp == 0) continue;
47             int rid = rev[id];
48             cap[id] -= temp;
49             cap[rid] += temp;
50             return temp;
51         }
52     }
53     return 0;
54 }
55 int maxflow() {
56     int flow = 0;
57     while(bfs()) {
58         memset(ptr, 0, sizeof(ptr));
59         while(true) {
60             int temp = dfs(s, inf);
61             if(temp == 0) break;
62             flow += temp;
63         }
64     }
65     return flow;
66 }

```

6.11 maximum cardinality matching

```

1 // maximum matching
2 // Edmonds' Blossom algorithm

```

```

3 // O(VEa(E,V))
4 class graph {
5     int n;
6     vector<int> p, d, a, c1, c2;
7     // (alternating tree), (unvisited: -1,
8     // even: 0, odd: 1), (auxiliary for lca
9     //), (cross edge)
10    int label{0};
11    /* DSU */
12    vector<int> dsu_p, dsu_sz, dsu_b;
13    void dsu_reset() {
14        fill(dsu_p.begin(), dsu_p.end(), -1)
15        ;
16        fill(dsu_sz.begin(), dsu_sz.end(),
17            1);
18        iota(dsu_b.begin(), dsu_b.end(), 0);
19    }
20    int find(int x) { // collapsing find
21        return dsu_p[x] == -1 ? x : dsu_p[x]
22            = find(dsu_p[x]);
23    }
24    int base(int x) { return dsu_b[find(x)];
25    }
26    void contract(int x, int y) { //
27        weighted union
28        auto rx{find(x)}, ry{find(y)};
29        if (rx == ry) return;
30        auto b{dsu_b[rx]}; // treat x's base
31        // as new base
32        if (dsu_sz[rx] < dsu_sz[ry]) swap(rx
33            , ry);
34        dsu_p[ry] = rx, dsu_sz[rx] += dsu_sz
35            [ry], dsu_b[rx] = b;
36    }
37    //*****
38    queue<int> qu{}; // only even vertices
39    void handle_one_side(int x, int y, int b
40        ) {
41        for (int v{base(x)}; v != b; v =
42            base(p[v])) {
43            c1[v] = x, c2[v] = y, contract(b
44                , v);
45            if (d[v] == 1) qu.push(v); //
46                odd vertex -> even vertex
47        }
48    }
49    int lca(int u, int v) {
50        ++label; // use next number in order
51        // not to clear a
52        while (true) {
53            if (a[u] == label) return u;
54            a[u] = label;
55            if (p[u] != -1) u = base(p[u]);
56            swap(u, v);
57        }
58    }
59    void augment(int r, int y) {
60        if (r == y) return;
61        if (d[y] == 0) { // even vertex ->
62            odd vertex
63            // check d[y] == 0 instead of d[
64                p[y]] == 1, so (m[y], y) is
65                in the blossom
66            augment(m[y], m[c1[y]]);
67            augment(r, m[c2[y]]);
68        }
69    }
70 }

```

```

71 m[c1[y]] = c2[y], m[c2[y]] = c1[
72     y];
73 } else {
74     int x{p[y]};
75     augment(r, m[x]);
76     m[x] = y, m[y] = x;
77 }
78 }
79 bool bfs(int r) {
80     dsu_reset();
81     fill(d.begin(), d.end(), -1);
82
83     qu = {}, qu.push(r), d[r] = 0;
84     while (!qu.empty()) {
85         int x{qu.front()}; qu.pop();
86         for (auto& y : adj[x]) {
87             if (base(x) == base(y))
88                 continue;
89             if (d[y] == -1) {
90                 p[y] = x, d[y] = 1;
91                 if (m[y] == -1) { //
92                     augmenting path
93                     augment(-1, y);
94                     return true;
95                 } else {
96                     p[m[y]] = y, d[m[y]]
97                         = 0;
98                     qu.push(m[y]);
99                 }
100             } else if (d[base(y)] == 0) {
101                 // blossom
102                 int b{lca(base(x), base(
103                     y))};
104                 handle_one_side(x, y, b)
105                 ;
106                 handle_one_side(y, x, b)
107                 ;
108             }
109         }
110     }
111     return false;
112 }
113 public:
114     vector<vector<int>>> adj;
115     vector<int> m;
116     explicit graph(int _n) : n{_n}, p(n, -1)
117         , d(n), a(n), c1(n), c2(n), dsu_p(n)
118         , dsu_sz(n), dsu_b(n), adj(n), m(n,
119             -1) {}
120     int Edmonds() {
121         int c{0};
122         for (int v{0}; v < n; ++v)
123             if (m[v] == -1 && bfs(v)) ++c;
124         return c;
125     }
126 };
127 // usage example:
128 int main() {
129     int n = 5;
130     graph g(n);
131     auto add = [&](int u, int v) {
132         g.adj[u].push_back(v);
133         g.adj[v].push_back(u);
134     };
135 }

```

```

136 // sample graph
137 add(0, 1);
138 add(0, 2);
139 add(1, 2); // triangle (0,1,2)
140 add(2, 3);
141 add(3, 4);
142
143 int match_size = g.Edmonds();
144 cout << "Maximum matching size: " <<
145     match_size << "\n";
146 for (int i = 0; i < n; ++i)
147     if (g.m[i] != -1 && i < g.m[i])
148         cout << i << " - " << g.m[i] <<
149             "\n";
150 }

```

6.12 topological sort

```

1 // topological sort 1
2 optional<vector<int>>> top_sort(vector<vector
3     <int>>& adj) {
4     vector<int> res{};
5     int n{static_cast<int>(adj.size())};
6     vector<int> cnt(n, 0); // predecessor
7     // count
8     for (int u = 0; u < n; ++u)
9         for (auto& v : adj[u]) ++cnt[v];
10
11     queue<int> qu{};
12     for (int u = 0; u < n; ++u) if (!cnt[u])
13         qu.push(u);
14     while (!qu.empty()) {
15         auto u = qu.front();
16         qu.pop();
17         res.push_back(u);
18
19         for (auto& v : adj[u])
20             if (--cnt[v] == 0) qu.push(v);
21     }
22     if (res.size() != adj.size()) return
23         nullopt;
24     return res;
25 }

```

6.13 Floyd's algorithm

```

1 // Floyd's Cycle-Finding Algorithm
2 // Given a function succ(x) that returns the
3 // successor of x,
4 a = succ(x);
5 b = succ(succ(x));
6 while (a != b) {
7     a = succ(a);
8     b = succ(succ(b));
9 }
10 // Find the first element in the cycle
11 a = x;
12 while (a != b) {

```

```

13     a = succ(a);
14     b = succ(b);
15 }
16 first = a;
17
18 // Find the length of the cycle
19 length = 1;
20 b = succ(a);
21 while (a != b) {
22     b = succ(b);
23     length = length + 1;
24 }
25
26 // Finding the starting point of the cycle
  from any node
27 int n;
28 vector<int> adj, res;
29 vector<vector<int>> radj;
30
31 void fill_radj(int u) {
32     for (int v : radj[u]) {
33         if (res[v] == -1) {
34             res[v] = res[u];
35             fill_radj(v);
36         }
37     }
38 }
39
40 void floyd(int x) {
41     int y = x;
42     do { // find a cycle using Floyd's
43         algorithm
44         x = adj[x];
45         y = adj[adj[y]];
46     } while (y != x);
47     do { // set res[x] = x for all x along
48         cycle
49         res[x] = x;
50         x = adj[x];
51     } while (y != x);
52     do { // set res'es for all x not along
53         cycle
54         fill_radj(x);
55         x = adj[x];
56     } while (y != x);
57 }
58
59 void solve() {
60     cin >> n;
61     adj.resize(n);
62     radj.assign(n, {});
63     res.assign(n, -1);
64
65     for (auto& e : adj) {
66         cin >> e;
67         e--;
68     }
69
70     for (int i = 0; i < n; ++i) {
71         radj[adj[i]].push_back(i);
72     }
73
74     for (int i = 0; i < n; ++i) {
75         if (res[i] == -1) {
76             floyd(i);
77         }
78     }
79 }

```

```

75     }
76
77     for (int i = 0; i < n; ++i) {
78         cout << res[i] + 1 << " ";
79     }
80     cout << "\n";
81 }

```

6.14 stable matching

```

1 // Stable Marriage Problem (stable matching)
2 // Gale-Shapley algorithm
3 class bipartiteGraph {
4 public:
5     int n;
6     vector<vector<int>> px, py;
7     vector<int> mx, my;
8
9     bipartiteGraph(int _n) : n(_n), px(n,
10         vector<int>(n)), py(n, vector<int>(n
11         )), mx(n, -1), my(n, -1) {}
12
13     void Gale_Shapley() {
14         queue<int> qu{};
15         vector<priority_queue<pair<int, int>
16             >>> pq(n);
17         for (int x{0}; x < n; ++x) {
18             qu.push(x);
19             for (int y{0}; y < n; ++y) pq[x
20                 ].emplace(px[x][y], y);
21         }
22
23         while (!qu.empty()) {
24             auto x{qu.front()}; qu.pop();
25
26             int y;
27             do {
28                 y = pq[x].top().second; pq[x
29                     ].pop();
30             } while (my[y] != -1 && py[y][x]
31                 <= py[y][my[y]]);
32
33             if (my[y] != -1) mx[my[y]] = -1,
34                 qu.push(my[y]); // y
35                 prefers x to my[y]
36             mx[x] = y, my[y] = x;
37         }
38     }
39
40     int main() {
41         int n = 3;
42         bipartiteGraph g(n);
43
44         // Men preferences (from most to Least
45         preferred)
46         vector<vector<int>> men = {
47             {0, 1, 2},
48             {1, 2, 0},
49             {1, 0, 2}
50         };
51
52         // Women preferences (from most to Least
53         preferred)

```

```

44     vector<vector<int>> women = {
45         {1, 0, 2},
46         {2, 1, 0},
47         {0, 1, 2}
48     };
49
50     // Convert orders to scores (higher is
51     better)
52     for (int x = 0; x < n; ++x)
53         for (int pos = 0; pos < n; ++pos)
54             g.px[x][men[x][pos]] = n - pos;
55
56     for (int y = 0; y < n; ++y)
57         for (int pos = 0; pos < n; ++pos)
58             g.py[y][women[y][pos]] = n - pos;
59
60     g.Gale_Shapley();
61
62     // Output matches: man x is matched to
63     woman g.mx[x]
64     for (int x = 0; x < n; ++x)
65         cout << x << " " << g.mx[x] << '\n';
66
67     return 0;
68 }

```

6.15 Bellman-Ford

```

1 // Bellman-Ford algorithm
2 template<typename T>
3 optional<vector<optional<T>>> Bellman_Ford(
4     const vector<vector<pair<int, T>>>& adj,
5     int s) {
6     const auto& n{adj.size()};
7     vector<optional<T>> d(n, nullopt);
8     d[s] = 0;
9
10    queue<int> qu{}, qu2{};
11    vector<bool> in(n, false), in2(n, false);
12
13    qu.push(s), in[s] = true;
14    for (int i{0}; i < n; ++i) { // at most
15        n-1 edges
16        while (!qu.empty()) {
17            int u{qu.front()};
18            qu.pop(), in[u] = false;
19            for (auto& [v, w] : adj[u])
20                if (!d[v] || d[v] > d[u].
21                    value() + w) { // relax
22                    d[v] = d[u].value() + w;
23                    if (!in2[v]) qu2.push(v),
24                        in2[v] = true;
25                }
26            qu.swap(qu2), in.swap(in2);
27        }
28        if (qu.empty()) return d;
29        return nullopt; // if negative cycle
30    }
31 }

```

7 Number Theory

7.1 Linear Sieve

```

1 // Calculate the smallest divisor of
  integers in [2, maxn) in O(maxn)
2 vector<int> min_div{};
3 constexpr int maxn = 40000 + 10;
4
5 vector<int> v(maxn), p;
6
7 for (int i = 2; i < maxn; ++i) {
8     if (!v[i]) {
9         v[i] = i;
10        p.push_back(i);
11    }
12    for (int j = 0; p[j] * i < maxn; ++j)
13        {
14            v[p[j] * i] = p[j];
15            if (p[j] == v[i]) break;
16        }
17
18    return v;
19 }()};

```

7.2 C(n,m)

```

1 ll Cnm(ll n, ll m) {
2     if (m > n / 2) m = n - m;
3     ll r{1};
4     for (ll i{1}, j{n}; i <= m; ++i, --j) r
5         *= j, r /= i;
6     return r;
7 }

```

7.3 Euler Totient 1 to n

```

1 void phi_1_to_n(int n) {
2     vector<int> phi(n + 1);
3     for (int i = 0; i <= n; i++)
4         phi[i] = i;
5
6     for (int i = 2; i <= n; i++) {
7         if (phi[i] == i) {
8             for (int j = i; j <= n; j += i)
9                 phi[j] -= phi[j] / i;
10        }
11    }
12 }

```


7.4 derangement (Principle of Inclusion-Exclusion)

```
1 // 1. Principle of Inclusion-Exclusion
2 //  $n! = n! * \sum_{k=0}^n [((-1)^k) / (k!)]$ 
3
4 mint c = 1;
5 for (int i = 1; i <= n; i++) {
6     c = (c * i) + (i % 2 == 1 ? -1 : 1);
7     cout << c.val() << ' ';
8 }
```

7.5 matrix template (with fast power)

```
1 template<class T> struct Matrix {
2     T **mat; int a, b;
3
4     Matrix() : a(0), b(0) {}
5     Matrix(int a_, int b_) : a(a_), b(b_) {
6         int i, j;
7         mat = new T*[a];
8         for (i = 0; i < a; ++i) {
9             mat[i] = new T[b];
10            for (j = 0; j < b; ++j) {
11                mat[i][j] = 0;
12            }
13        }
14    }
15    Matrix(const vector<vector<T>>& vt) {
16        int i, j;
17
18        *this = Matrix((int)vt.size(), (int)
19            vt[0].size());
20        for (i = 0; i < a; ++i) {
21            for (j = 0; j < b; ++j) {
22                mat[i][j] = vt[i][j];
23            }
24        }
25    }
26    Matrix operator*(const Matrix& m) {
27        int i, j, k;
28
29        assert(b == m.a);
30        Matrix r(a, m.b);
31        for (i = 0; i < a; ++i) {
32            for (j = 0; j < m.b; ++j) {
33                for (k = 0; k < b; ++k) {
34                    r.mat[i][j] += mul(mat[i][k], m.mat[k][j]);
35                }
36            }
37        }
38        return r;
39    }
40    Matrix& operator*=(const Matrix& m) {
41        return *this = (*this) * m;
42    }
43 }
```

```
43 friend Matrix power(Matrix m, long long
44     p) {
45     int i;
46
47     assert(m.a == m.b);
48     Matrix r(m.a, m.b);
49     for (i = 0; i < m.a; ++i) {
50         r.mat[i][i] = 1;
51     }
52     for (; p > 0; p >>= 1, m *= m) {
53         if (p & 1) {
54             r *= m;
55         }
56     }
57     return r;
58 }
59
60 int main() {
61     Matrix<int> mat(adj);
62     mat = power(mat, k); // mat^k
63     cout << mat.mat[i][j];
64 }
```

7.6 Sieve of Eratosthenes (with big num)

```
1 const int MX = 100000;
2 bool np[MX + 1];
3 vector<int> prime_numbers;
4
5 int init = []() {
6     np[0] = np[1] = true;
7     for (int i = 2; i <= MX; i++) {
8         if (!np[i]) {
9             prime_numbers.push_back(i);
10            for (int j = i; j <= MX / i; j
11                ++){ // 避免溢出的写法
12                np[i * j] = true;
13            }
14        }
15    }
16    return 0;
17 }();
18
19 bool is_prime(long long n) {
20     if (n <= MX) {
21         return !np[n];
22     }
23     for (long long p : prime_numbers) {
24         if (p * p > n) {
25             break;
26         }
27         if (n % p == 0) {
28             return false;
29         }
30     }
31     return true;
32 }
```

7.7 mod inv

```
1 // Modular inverse when mod is prime
2 long long mod_inverse(long long a, long long
3     mod) {
4     return power_mod(a, mod - 2, mod); //
5     // Fermat's Little Theorem
6 }
```

7.8 derangement (DP)

```
1 // 2. DP
2 //  $!n = (n - 1) * (! (n - 1) + ! (n - 2))$ ,
3 // with  $!0 = 1, !1 = 0$ 
4
5 mint a = 1, b = 0;
6 cout << 0 << ' ';
7
8 for (int i = 2; i <= n; i++) {
9     mint c = (i - 1) * (a + b);
10    cout << c.val() << ' ';
11    a = b;
12    b = c;
13 }
```

7.9 Euler Totient

```
1 int phi(int n) {
2     int result = n;
3     for (int i = 2; i * i <= n; i++) {
4         if (n % i == 0) {
5             while (n % i == 0)
6                 n /= i;
7             result -= result / i;
8         }
9     }
10    if (n > 1)
11        result -= result / n;
12    return result;
13 }
```

7.10 fast power

```
1 /* Iterative Function to calculate (a^b) %
2     mod in O(log b) */
3 long long power_mod(long long a, long long b
4     , long long mod) {
5     long long res{1};
6     while (b > 0) {
7         if (b & 1) res = res * a % mod;
8         b >>= 1;
9         a = (a * a) % mod;
10    }
11    return res;
12 }
```

7.11 first and second mex

```
1 // Calculate first and second MEX
2 pair<int, int> calculate_mexes(vector<int>&
3     nums) {
4     int n = nums.size();
5     vector<bool> seen(n + 2, false);
6
7     for (int num : nums) {
8         if (num >= 0 && num < seen.size()) {
9             seen[num] = true;
10        }
11    }
12
13    int first_mex = -1;
14    int second_mex = -1;
15
16    for (int i = 0; i < seen.size(); ++i) {
17        if (!seen[i]) {
18            if (first_mex == -1) {
19                first_mex = i;
20            } else {
21                second_mex = i;
22                break;
23            }
24        }
25    }
26
27    return {first_mex, second_mex};
28 }
```

7.12 Catalan Number

```
1 // Catalan Number
2 //  $C(n) = 1/(n+1) * (2n \text{ choose } n) = (2n)! /$ 
3 //  $((n+1)! * n!)$ 
4
5 ll catalan(int n) {
6     return fact[2 * n] * invf[n + 1] % mod *
7     invf[n] % mod;
8 }
```

7.13 Chinese Remainder Theorem

```
1 // coprime (p^k)
2 pair<ll, ll> CRT(const vector<pair<ll, ll>>&
3     congruences) {
4     ll M = 1, sol = 0;
5     for (auto& [m, a] : congruences) M *= m;
6     for (auto& [m, a] : congruences) {
7         ll x = M / m, y = MI(x, m);
8         sol = MA(sol, MM(MM(a, x, M), y, M),
9             M);
10    }
11    return {M, sol};
12 }
13
14 // example usage
15 int main() {
```

```

14 // x ≡ 2 (mod 3)
15 // x ≡ 3 (mod 5)
16 // x ≡ 2 (mod 7)
17 vector<pair<ll, ll>> congruences = {{3,
18     2}, {5, 3}, {7, 2}};
19 auto [M, sol] = CRT(congruences);
20 cout << "M: " << M << ", x: " << sol <<
21 endl; // M: 105, x: 23
22 return 0;
23 }

```

7.14 mod inv (not prime)

```

1 /* exists when a and mod are coprime */
2 /* but mod is not prime */
3 long long MI(long long a, long long mod) {
4     return power_mod(a, euler_phi(mod) - 1,
5         mod);
6 }

```

7.15 mod inv (not coprime)

```

1 /* a and mod are not coprime */
2 long long MI(long long a, long long mod) {
3     long long d, x, y;
4     extEcu(a, mod, d, x, y);
5     return d == 1 ? (x + mod) % mod : -1;
6 }

```

7.16 inclusion-exclusion principle

```

1 /*
2 inclusion-exclusion principle:  $|U \setminus A| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots$ 
3
4 if  $f(n) = \sum g(d)$  for all  $d|n$ 
5 then  $g(n) = \sum \mu(d) * f(n/d)$  for all  $d|n$ 
6
7 where  $\mu(d)$  is the mobius function defined as
8 :
9  $|U \setminus A| = \sum \mu(d) * |A_d|$ 
10 where  $A_d = \{x \in U : d|x\}$ 
11  $\mu(d) = 1$  if  $d$  is a product of even number of
12 distinct primes
13  $\mu(d) = -1$  if  $d$  is a product of odd number of
14 distinct primes
15  $\mu(d) = 0$  if  $d$  has a squared prime factor
16 */
17 problem: In number theory we call an integer
18 square-free if it is not divisible by a
19 perfect square, except 1. You have to
20 count them!

```

```

18 square-free number: An integer n is square-
19 free if no p^2 divides n.
20
21 /*
22 const int VALMAX = 1e7;
23
24 int mobius[VALMAX];
25
26 int main() {
27     int test_num;
28     cin >> test_num;
29
30     mobius[1] = -1;
31     for (int i = 1; i < VALMAX; i++) {
32         if (mobius[i]) {
33             mobius[i] = -mobius[i];
34             for (int j = 2 * i; j < VALMAX; j += i)
35                 { mobius[j] += mobius[i]; }
36         }
37     }
38
39     for (int t = 0; t < test_num; t++) {
40         long long n;
41         long long ans = 0;
42         cin >> n;
43         for (int i = 1; 1LL * i * i <= n; i++) {
44             ans += mobius[i] * n / ((long long)i *
45                 i);
46         }
47         cout << ans << '\n';
48     }
49 }
50
51 /*
52 problem: Given N cows (2 <= N <= 50,000),
53 each listing five favorite ice cream
54 flavors,
55 where two cows are compatible if they share
56 at least one flavor,
57 determine the number of cow pairs that are
58 not compatible.
59
60 ans = n * (n - 1) / 2 -  $\sum |A_i| + \sum |A_i \cap A_j| - \dots$ 
61 where  $A_i$  is the set of cows that like
62 flavor i
63
64 */
65 void solve() {
66     int n;
67     cin >> n;
68     vector<array<int, 5>> vc(n);
69
70     map<vector<int>, int> subsets;
71     for (int i = 0; i < n; ++i) {
72         for (int j = 0; j < 5; ++j) {
73             cin >> vc[i][j];
74         }
75
76         sort(vc[i].begin(), vc[i].end());
77         for (int mask = 1; mask < (1<<5); ++
78             mask) {
79             vector<int> subset;
80             for (int b = 0; b < 5; ++b) {
81                 if (mask & (1<<b)) {
82                     subset.push_back(vc[i][b
83                         ]);
84                 }
85             }
86         }
87     }
88 }

```

```

73     }
74     }
75     subsets[subset]++;
76 }
77
78 int ans = n * (n - 1) / 2;
79 for (const auto& [subset, freq] :
80     subsets) {
81     ans -= (subset.size() % 2 == 1 ? 1 :
82         -1) * freq * (freq - 1) / 2;
83 }
84 cout << ans << "\n";
85 }

```

7.17 C(n,k) mod inverse

```

1 fac[0] = 1;
2 for (int i = 1; i <= n; ++i) {
3     fac[i] = fac[i - 1] * i % MOD;
4 }
5
6 inv_fac[n] = power_mod(fac[n], MOD - 2, MOD)
7 ;
8 for (int i = n - 1; i >= 0; --i) {
9     inv_fac[i] = inv_fac[i + 1] * (i + 1) %
10     MOD;
11 }
12
13 // C(n, k) = fac[n] * inv_fac[k] * inv_fac[n
14 - k];

```

7.18 Linear Diophantine 丢番圖

```

1 // ax + by = c
2 // x0 = x * (c / gcd(a, b))
3 // y0 = y * (c / gcd(a, b))
4 // x = x0 + k * (b / gcd(a, b))
5 // y = y0 - k * (a / gcd(a, b))
6 // k is any integer
7
8 t11 extendedEuclid(ll a, ll b) {
9     if (b == 0) return {a, 1, 0};
10     ll d, x1, y1;
11     tie(d, x1, y1) = extendedEuclid(b, a % b
12         );
13     return {d, y1, x1 - (a / b) * y1};
14 }
15
16 p11 linearDiophantine(ll a, ll b, ll c) {
17     ll d, x, y;
18     tie(d, x, y) = extendedEuclid(a, b);
19     if (c % d != 0) return {-1, -1}; // no
20     solution
21     x *= c / d;
22     y *= c / d;
23     return {x, y};
24 }
25
26 // Example usage

```

```

25 int main() {
26     ll a = 15, b = 10, c = 35;
27     p11 res = linearDiophantine(a, b, c);
28     if (res.first == -1 && res.second == -1)
29         {
30             cout << "No solution exists" << endl;
31         }
32     else {
33         cout << "One solution is x = " <<
34             res.first << ", y = " <<
35             res.second << endl;
36     }
37     return 0;
38 }
39 ax + by = c

```

7.19 擴展歐基里德

```

1 /* solve x, y for ax + by = gcd(a, b) = g */
2 template<typename T>
3 void extEcu(T a, T b, T &g, T &x, T &y) {
4     if (b) extEcu(b, a % b, g, y, x), y -= x
5         * (a / b);
6     else g = a, x = 1, y = 0;
7 }

```

7.20 Sieve of Eratosthenes

```

1 void sieve(vector<int>& primes) {
2     vector<int> is_prime(INF + 1, 0);
3     is_prime[0] = 1;
4     is_prime[1] = 1;
5
6     int sq = sqrt(INF);
7
8     for (int i = 2; i <= sq; ++i) {
9         if (!is_prime[i]) {
10             primes.push_back(i);
11             for (int j = i * i; j <= INF; j
12                 += i) {
13                 is_prime[j] = 1;
14             }
15         }
16     }
17 }

```

7.21 builtin

```

1 // __builtin_ctz
2 // 這個函數作用是返回輸入數二進制表示從最低
3 // 位開始 (右起) 的連續的0的個數; 如果傳入0
4 // 則行為未定義。
5 int __builtin_ctz (unsigned int x)
6
7 // __builtin_clz

```

```

6 // 這個函數作用是返回輸入數二進制表示從最高
  位開始 (左起) 的連續的0的個數; 如果傳入0
  則行為未定義。
7 int __builtin_clz (unsigned int x)
8
9 // __builtin_ffs
10 // 這個函數作用是返回輸入數二進制表示的最低
   非0位的下標, 下標從1開始計數; 如果傳入0
   則返回0
11 int __builtin_ffs (unsigned int x)
12
13 // __builtin_popcount
14 // 這個函數作用是返回輸入數二進制表示中1的個
   數。
15 int __builtin_popcount (unsigned int x)
16
17 // __builtin_parity
18 // 這個函數作用是返回輸入數二進制表示中1的個
   數的奇偶性, 1表示奇數個1, 0表示偶數個1。
19 int __builtin_parity (unsigned int x)

```

7.22 C(n,k) DP

```

1 long long binomial(long long n, long long k,
  long long p) {
2 // dp[i][j] = iCj
3 vector<vector<long long>> dp(n + 1,
  vector<long long>(k + 1, 0));
4
5 for (int i = 0; i <= n; ++i) {
6 dp[i][0] = 1;
7 if (i <= k) dp[i][i] = 1;
8 }
9
10 for (int i = 0; i <= n; ++i) {
11 for (int j = 1; j <= min(i, k); ++j)
12 {
13 if (i != j) {
14 dp[i][j] = (dp[i - 1][j - 1]
15 + dp[i - 1][j]) % p;
16 }
17 }
18 return dp[n][k];
19 }

```

8 Range Queries

8.1 subarray maximum sum queries

```

1 // CSES: Subarray Sum Queries
2
3 const int maxn = 2e5;
4 int n = 0;
5 int t[maxn << 1];

```

```

6 void build() {
7 for (int i = 0; i < n; ++i) {
8 t[i + n] = 0;
9 }
10
11 for (int i = n - 1; i >= 1; --i) {
12 t[i] = t[i << 1] + t[i << 1 | 1];
13 }
14 }
15
16 void update(int idx, int val) {
17 idx += n;
18 if (t[idx] == val) return;
19 t[idx] = val;
20 for (idx >>= 1; idx >= 1; idx >>= 1) {
21 t[idx] = t[idx << 1] + t[idx << 1 |
22 1];
23 }
24 }
25
26 int query(int l, int r) {
27 int ans = 0;
28 for (l += n, r += n; l < r; l >>= 1, r
29 >>= 1) {
30 if (l & 1) ans += t[l++];
31 if (r & 1) ans += t[--r];
32 }
33 return ans;
34 }
35
36 void solve() {
37 int q;
38 cin >> q;
39 n = 0;
40 vector<pii> op(q);
41 map<int, int> mp;
42 vector<int> to_val;
43
44 memset(t, 0, sizeof(t));
45
46 for (int i = 0; i < q; ++i) {
47 char c;
48 cin >> c >> op[i].second;
49 if (c == 'I') {
50 op[i].first = 0;
51 mp[op[i].second] = 1;
52 }
53 else if (c == 'D') {
54 op[i].first = 1;
55 mp[op[i].second] = 1;
56 }
57 else if (c == 'K') {
58 op[i].first = 2;
59 }
60 else {
61 op[i].first = 3;
62 }
63 }
64
65 for (auto& x : mp) {
66 x.second = n;
67 to_val.push_back(x.first);
68 n++;
69 }
70 build();

```

```

71 for (pii o : op) {
72 if (o.first == 0) {
73 int idx = mp[o.second];
74 update(idx, 1);
75 }
76 if (o.first == 1) {
77 int idx = mp[o.second];
78 update(idx, 0);
79 }
80 if (o.first == 2) {
81 int k = o.second;
82 int ans = - 1;
83 int l = 0, r = n - 1;
84 while (l <= r) {
85 int m = l + (r - l) / 2;
86 int rk = query(0, m + 1);
87 if (rk >= k) {
88 ans = m;
89 r = m - 1;
90 }
91 else {
92 l = m + 1;
93 }
94 }
95 if (ans == -1) cout << "invalid\
96 n";
97 else cout << to_val[ans] << "\n"
98 ;
99 }
100 if (o.first == 3) {
101 auto it = mp.lower_bound(o.
102 second);
103 if (it == mp.begin() && (it ==
104 mp.end() || o.second <= it->
105 first)) {
106 cout << "0\n";
107 }
108 else if (it == mp.end()) {
109 cout << query(0, n) << "\n";
110 }
111 else {
112 cout << query(0, it->second)
113 << "\n";
114 }
115 }
116 }
117 }
118 }
119 }
120 }

```

8.2 find first equal or greater

```

1 // CSE: Hotel Queries
2
3 int n, m;
4 int sz = 1;
5 const int maxn = 2e6;
6 int t[maxn << 1];
7
8 void build(const vector<int>& a) {
9 for (int i = 0; i < n; ++i) {
10 t[i + sz] = a[i];
11 }

```

```

12 for (int i = sz - 1; i >= 1; --i) {
13 t[i] = max(t[i << 1], t[i << 1 | 1]);
14 }
15 }
16
17 void update(int i, int val) {
18 i += sz;
19 t[i] = val;
20 for (i >>= 1; i >= 1; i >>= 1) {
21 t[i] = max(t[i << 1], t[i << 1 | 1]);
22 }
23 }
24
25 int query(int r) {
26 if (t[1] < r) return 0;
27
28 int i = 1;
29 while (i < sz) {
30 if (t[i << 1] >= r) i = i << 1;
31 else i = i << 1 | 1;
32 }
33 return i - sz + 1;
34 }
35
36 void solve() {
37 cin >> n >> m;
38 vector<int> a(n);
39
40 for (int i = 0; i < n; ++i) {
41 cin >> a[i];
42 }
43
44 while (sz < n) sz <= 1;
45 memset(t, 0, sizeof(t));
46 build(a);
47
48 int r;
49 while (m--) {
50 cin >> r;
51 int idx = query(r);
52 cout << idx << " ";
53 if (idx) {
54 a[idx - 1] -= r;
55 update(idx - 1, a[idx - 1]);
56 }
57 }
58 cout << "\n";
59 }

```

8.3 find k-th and count less than

```

1 /*
2 SPOJ: ORDERSET - Order statistic set
3
4 In this problem, you have to maintain a
  dynamic set of numbers which support the
  two fundamental operations
5
6 INSERT(S,x): if x is not in S, insert x into
  S
7 DELETE(S,x): if x is in S, delete x from S

```

```

8 and the two type of queries
9
10 K-TH(S) : return the k-th smallest element
    of S
11 COUNT(S,x): return the number of elements of
    S smaller than x
12
13 */
14 const int maxn = 2e5;
15 int n = 0;
16 int t[maxn << 1];
17
18 void build() {
19     for (int i = 0; i < n; ++i) {
20         t[i + n] = 0;
21     }
22     for (int i = n - 1; i >= 1; --i) {
23         t[i] = t[i << 1] + t[i << 1 | 1];
24     }
25 }
26
27 void update(int idx, int val) {
28     idx += n;
29     if (t[idx] == val) return;
30     t[idx] = val;
31     for (idx >= 1; idx >= 1; idx >= 1) {
32         t[idx] = t[idx << 1] + t[idx << 1 | 1];
33     }
34 }
35
36 int query(int l, int r) {
37     int ans = 0;
38     for (l += n, r += n; l < r; l >= 1, r >= 1) {
39         if (l & 1) ans += t[l++];
40         if (r & 1) ans += t[--r];
41     }
42     return ans;
43 }
44
45 void solve() {
46     int q;
47     cin >> q;
48     n = 0;
49     vector<pii> op(q);
50     map<int, int> mp;
51     vector<int> to_val;
52
53     memset(t, 0, sizeof(t));
54
55     for (int i = 0; i < q; ++i) {
56         char c;
57         cin >> c >> op[i].second;
58         if (c == 'I') {
59             op[i].first = 0;
60             mp[op[i].second] = 1;
61         }
62         else if (c == 'D') {
63             op[i].first = 1;
64             mp[op[i].second] = 1;
65         }
66         else if (c == 'K') {
67             op[i].first = 2;
68         }
69         else {

```

```

70         op[i].first = 3;
71     }
72 }
73
74 for (auto& x : mp) {
75     x.second = n;
76     to_val.push_back(x.first);
77     n++;
78 }
79
80 build();
81
82 for (pii o : op) {
83     if (o.first == 0) {
84         int idx = mp[o.second];
85         update(idx, 1);
86     }
87     if (o.first == 1) {
88         int idx = mp[o.second];
89         update(idx, 0);
90     }
91     if (o.first == 2) {
92         int k = o.second;
93         int ans = -1;
94         int l = 0, r = n - 1;
95         while (l <= r) {
96             int m = l + (r - l) / 2;
97             int rk = query(0, m + 1);
98             //printf("(m, rk): %lld, %lld\n", m, rk);
99             if (rk >= k) {
100                 ans = m;
101                 r = m - 1;
102             }
103             else {
104                 l = m + 1;
105             }
106         }
107         if (ans == -1) cout << "invalid\n";
108         else cout << to_val[ans] << "\n";
109     }
110     if (o.first == 3) {
111         auto it = mp.lower_bound(o.second);
112         if (it == mp.begin() && (it == mp.end() || o.second <= it->first)) {
113             cout << "0\n";
114         }
115         else if (it == mp.end()) {
116             cout << query(0, n) << "\n";
117         }
118         else {
119             cout << query(0, it->second) << "\n";
120         }
121     }
122 }
123 }
124 }

```

9 String

9.1 Z

```

1 // 計算並返回 z 數組，其中 z[i] = |LCP(s[i
    :], s)|
2 vector<int> calc_z(const string& s) {
3     int n = s.size();
4     vector<int> z(n);
5     int box_l = 0, box_r = 0;
6     for (int i = 1; i < n; i++) {
7         if (i <= box_r) {
8             z[i] = min(z[i - box_l], box_r -
                i + 1);
9         }
10        while (i + z[i] < n && s[z[i]] == s[
            i + z[i]]) {
11            box_l = i;
12            box_r = i + z[i];
13            z[i]++;
14        }
15    }
16    z[0] = n;
17    return z;
18 }

```

9.2 manacher

```

1 class Solution {
2 public:
3     int countSubstrings(string s) {
4         int l1 = s.size(), l2 = l1 * 2 + 1;
5         string ch = "#";
6         for (char c : s) {
7             ch = ch + c + "#";
8         }
9
10        int c = 0, r = 0, cnt = 0;
11        vector<int> p(l2);
12        for (int i = 0; i < l2; i++) {
13            p[i] = (i < r) ? min(p[2 * c - i
                ], r - i) : 1;
14            while (i + p[i] < l2 && i - p[i]
                >= 0 && ch[i + p[i]] == ch[i
                    - p[i]]) p[i]++;
15            if (i + p[i] > r) {
16                r = i + p[i];
17                c = i;
18            }
19            int l = p[i] - 1;
20            if (l % 2 == 0) cnt += 1 / 2;
21            else cnt += 1 / 2 + 1;
22        }
23        return cnt;
24    }
25 };

```

9.3 AC 自動機

```

1 template<char L='a',char R='z'>
2 class ac_automaton{
3     struct joe{
4         int next[R-L+1], fail, efl, ed, cnt_dp, vis;
5         joe():ed(0),cnt_dp(0),vis(0){
6             for(int i=0;i<=R-L;++i)next[i]=0;
7         }
8     };
9     public:
10        std::vector<joe> S;
11        std::vector<int> q;
12        int qs,qe,vt;
13        ac_automaton():S(1),qs(0),qe(0),vt(0){}
14        void clear(){
15            q.clear();
16            S.resize(1);
17            for(int i=0;i<=R-L;++i)S[0].next[i]=0;
18            S[0].cnt_dp=S[0].vis=qs=qe=vt=0;
19        }
20        void insert(const char *s){
21            int o=0;
22            for(int i=0,id;s[i];++i){
23                id=s[i]-L;
24                if(!S[o].next[id]){
25                    S.push_back(joe());
26                    S[o].next[id]=S.size()-1;
27                }
28                o=S[o].next[id];
29            }
30            ++S[o].ed;
31        }
32        void build_fail(){
33            S[0].fail=S[0].efl=-1;
34            q.clear();
35            q.push_back(0);
36            ++qe;
37            while(qs!=qe){
38                int pa=q[qs++],id,t;
39                for(int i=0;i<=R-L;++i){
40                    t=S[pa].next[i];
41                    if(!t)continue;
42                    id=S[pa].fail;
43                    while(~id&&!S[id].next[i])id=S[id].fail;
44                    S[t].fail=~id?S[id].next[i]:0;
45                    S[t].efl=S[t].fail.ed?S[t].fail:
                        S[t].fail.efl;
46                    q.push_back(t);
47                    ++qe;
48                }
49            }
50        }
51        /*DP出每個前綴在字串s出現的次數並傳回所有
            字串被s匹配成功的次數O(N*M)*/
52        int match_0(const char *s){
53            int ans=0,id,p=0,i;
54            for(i=0;s[i];++i){
55                id=s[i]-L;
56                while(!S[p].next[id]&&p=S[p].fail;
57                    if(!S[p].next[id])continue;
58                p=S[p].next[id];
59                ++S[p].cnt_dp; /*匹配成功則它所有後綴都
                    可以被匹配(DP計算)*/
60            }
61            for(i=qe-1;i>=0;--i){

```

```

62     ans+=S[q[i]].cnt_dp*S[q[i]].ed;
63     if(~S[q[i]].fail)S[S[q[i]].fail].
        cnt_dp+=S[q[i]].cnt_dp;
64 }
65 return ans;
66 }
67 /*多串匹配走efl邊並傳回所有字串被s匹配成功
    的次數 $O(N*M^{1.5})$ */
68 int match_1(const char *s) const {
69     int ans=0, id, p=0, t;
70     for(int i=0; s[i]; ++i){
71         id=s[i]-L;
72         while(!S[p].next[id]&&p=S[p].fail;
73             if(!S[p].next[id]) continue;
74         p=S[p].next[id];
75         if(S[p].ed) ans+=S[p].ed;
76         for(t=S[p].efl; ~t; t=S[t].efl){
77             ans+=S[t].ed; /*因為都走efl邊所以保證
                匹配成功*/
78         }
79     }
80     return ans;
81 }
82 /*枚舉(s的子字串a)的所有相異字串各恰一次
    並傳回次數 $O(N*M^{(1/3)})$ */
83 int match_2(const char *s){
84     int ans=0, id, p=0, t;
85     ++vt;
86     /*把戳記vt+=1，只要vt沒溢位，所有S[p].
        vis==vt就會變成false
87     這種利用vt的方法可以 $O(1)$ 歸零vis陣列*/
88     for(int i=0; s[i]; ++i){
89         id=s[i]-L;
90         while(!S[p].next[id]&&p=S[p].fail;
91             if(!S[p].next[id]) continue;
92         p=S[p].next[id];
93         if(S[p].ed&&S[p].vis!=vt){
94             S[p].vis=vt;
95             ans+=S[p].ed;
96         }
97         for(t=S[p].efl; ~t&&S[t].vis!=vt; t=S[t].efl){
98             S[t].vis=vt;
99             ans+=S[t].ed; /*因為都走efl邊所以保證
                匹配成功*/
100         }
101     }
102     return ans;
103 }
104 /*把AC自動機變成真的自動機*/
105 void evolution(){
106     for(qs=1; qs!=qe;){
107         int p=q[qs++];
108         for(int i=0; i<=R-L; ++i)
109             if(S[p].next[i]==0) S[p].next[i]=S[S[p].fail].next[i];
110     }
111 }
112 };

```

9.4 KMP

```

1 // 在文本串 text 中查找模式串 pattern，返回
    所有成功匹配的位置 (pattern[0] 在 text
    中的下標)
2 vector<int> kmp(const string& text, const
    string& pattern) {
3     int m = pattern.size();
4     vector<int> pi(m);
5     int cnt = 0;
6     for (int i = 1; i < m; i++) {
7         char b = pattern[i];
8         while (cnt && pattern[cnt] != b) {
9             cnt = pi[cnt - 1];
10        }
11        if (pattern[cnt] == b) {
12            cnt++;
13        }
14        pi[i] = cnt;
15    }
16
17    vector<int> pos;
18    cnt = 0;
19    for (int i = 0; i < text.size(); i++) {
20        char b = text[i];
21        while (cnt && pattern[cnt] != b) {
22            cnt = pi[cnt - 1];
23        }
24        if (pattern[cnt] == b) {
25            cnt++;
26        }
27        if (cnt == m) {
28            pos.push_back(i - m + 1);
29            cnt = pi[cnt - 1];
30        }
31    }
32    return pos;
33 }

```

9.5 suffix array lcp

```

1 #define radix_sort(x,y){\
2     for(i=0;i<A;++i)c[i]=0;\
3     for(i=0;i<n;++i)c[x[y[i]]]++; \
4     for(i=1;i<A;++i)c[i]+=c[i-1]; \
5     for(i=n-1; ~i; --i)sa[--c[x[y[i]]]]=y[i]; \
6 }
7 #define AC(r,a,b)\
8     r[a] != r[b] || a+k==n || r[a+k] != r[b+k]
9 void suffix_array(const char *s, int n, int *
    sa, int *rank, int *tmp, int *c){
10     int A='z'+1, i, k, id=0;
11     for(i=0; i<n; ++i) rank[tmp[i]=i]=s[i];
12     radix_sort(rank, tmp);
13     for(k=1; id<n-1; k<=1){
14         for(id=0, i=n-k; i<n; ++i) tmp[id++] = i;
15         for(i=0; i<n; ++i)
16             if(sa[i]>=k) tmp[id++] = sa[i]-k;
17         radix_sort(rank, tmp);
18         swap(rank, tmp);
19         for(rank[sa[0]]=id=0, i=1; i<n; ++i)
20             rank[sa[i]] = id += AC(tmp, sa[i-1], sa[i]);
21         A=id+1;
22     }
}

```

```

23 }
24 //h:高度數組 sa:後綴數組 rank:排名
25 void suffix_array_lcp(const char *s, int len,
    int *h, int *sa, int *rank){
26     for(int i=0; i<len; ++i) rank[sa[i]]=i;
27     for(int i=0, k=0; i<len; ++i){
28         if(rank[i]==0) continue;
29         if(k)--k;
30         while(s[i+k]==s[sa[rank[i]-1]+k]) ++k;
31         h[rank[i]]=k;
32     }
33     h[0]=0; // h[k]=lcp(sa[k], sa[k-1]);
34 }

```

9.6 hash

```

1 #define MAXN 1000000
2 #define mod 1073676287
3 /*mod 必須要是質數*/
4 typedef long long T;
5 char s[MAXN+5];
6 T h[MAXN+5]; /*hash陣列*/
7 T h_base[MAXN+5]; /*h_base[n]=(prime^n)%mod*/
8 void hash_init(int len, T prime){
9     h_base[0]=1;
10    for(int i=1; i<=len; ++i){
11        h[i]=(h[i-1]*prime+s[i-1])%mod;
12        h_base[i]=(h_base[i-1]*prime)%mod;
13    }
14 }
15 T get_hash(int l, int r){ /*閉區間寫法，設編號
    為0 ~ Len-1*/
16     return (h[r+1]-(h[l]*h_base[r-l+1])%mod+
    mod)%mod;
17 }

```

9.7 minimal string rotation

```

1 int min_string_rotation(const string &s){
2     int n=s.size(), i=0, j=1, k=0;
3     while(i<n&&j<n&&k<n){
4         int t=s[(i+k)%n]-s[(j+k)%n];
5         ++k;
6         if(t){
7             if(t>0) i+=k;
8             else j+=k;
9             if(i==j) ++j;
10            k=0;
11        }
12    }
13    return min(i, j); //最小循環表示法起始位置
14 }

```

9.8 reverseBWT

```

1 const int MAXN = 305, MAXC = 'Z';
2 int ranks[MAXN], tots[MAXC], first[MAXC];
3 void rankBWT(const string &bw){
4     memset(ranks, 0, sizeof(int)*bw.size());
5     memset(tots, 0, sizeof(tots));
6     for(size_t i=0; i<bw.size(); ++i)
7         ranks[i] = tots[int(bw[i])]+1;
8 }
9 void firstCol(){
10    memset(first, 0, sizeof(first));
11    int totc = 0;
12    for(int c='A'; c<='Z'; ++c){
13        if(!tots[c]) continue;
14        first[c] = totc;
15        totc += tots[c];
16    }
17 }
18 string reverseBwt(string bw, int begin){
19     rankBWT(bw);
20     int i = begin; //原字串最後一個元素的位置
21     string res;
22     do{
23         char c = bw[i];
24         res = c + res;
25         i = first[int(c)] + ranks[i];
26     } while(i != begin);
27     return res;
28 }

```

10 Tree

10.1 Euler tour+RMQ

```

1 // Euler Tour Technique
2 class LCA {
3     const vector<vector<int>>& adj;
4     int n;
5     vector<int> d, first, euler{}, log2{};
6     vector<vector<int>> st{};
7     void dfs(int u, int w = -1, int dep = 0)
8     {
9         d[u] = dep;
10        first[u] = euler.size();
11        euler.push_back(u);
12        for (auto& v : adj[u]) {
13            if (v == w) continue;
14            dfs(v, u, dep + 1);
15            euler.push_back(u);
16        }
17    public:
18        LCA(const vector<vector<int>>& _adj, int
            root) : adj{_adj}, n{adj.size()}, d
            (n), first(n) {
19            dfs(root);
20
21            int tn{euler.size()};
22            log2.resize(tn + 1);
23            log2[1] = 0;
24            for (int i{2}; i <= tn; ++i) log2[i]
                = log2[i / 2] + 1;

```



```

25 st.assign(tn, vector<int>(log2[tn] + 1));
26 for (int i{tn - 1}; i >= 0; --i) {
27     st[i][0] = euler[i];
28     for (int j{1}; i + (1 << j) <= tn; ++j) {
29         auto& x{st[i][j - 1]};
30         auto& y{st[i + (1 << (j - 1))]};
31         st[i][j] = d[x] <= d[y] ? x : y;
32     }
33 }
34 }
35 }
36 int operator()(int u, int v) {
37     int l{first[u]}, r{first[v]};
38     if (l > r) swap(l, r);
39     ++r; // make the interval left closed right open
40
41     int j{log2[r - l]};
42     auto& x{st[l][j]};
43     auto& y{st[r - (1 << j)][j]};
44     return d[x] <= d[y] ? x : y;
45 }
46 };
47
48 int main() {
49     int n, q;
50     cin >> n >> q;
51     vector<vector<int>> adj(n);
52
53     for (int i = 1; i < n; ++i) {
54         int u;
55         cin >> u;
56         u--;
57         adj[u].pb(i);
58         adj[i].pb(u);
59     }
60
61     LCA lca(adj, 0);
62
63     while (q--) {
64         int u, v;
65         cin >> u >> v;
66         u--, v--;
67         cout << lca(u, v) + 1 << "\\n";
68     }
69     return 0;
70 }

```

10.2 HLD template

```

1 struct HLD {
2     int n, cur = 0;
3     vector<int> sz, top, dep, par, tin, tout, seq;
4     vector<vector<int>> adj;
5     HLD(int n = 0) {
6         init(n);
7     }
8     void init(int n_) {

```

```

9         n = n_;
10        sz.assign(n, 1);
11        top.assign(n, -1);
12        dep.assign(n, 0);
13        par.assign(n, -1);
14        tin.assign(n, 0);
15        tout.assign(n, 0);
16        seq.assign(n, 0);
17        adj.resize(n, {});
18    }
19    void add_edge(int u, int v) { adj[u].push_back(v); adj[v].push_back(u); }
20    void build(int root = 0) {
21        top[root] = root, dep[root] = 0, par[root] = -1;
22        dfs1(root), dfs2(root);
23    }
24    void dfs1(int u) {
25        if (auto it = find(adj[u].begin(), adj[u].end(), par[u]); it != adj[u].end()) {
26            adj[u].erase(it);
27        }
28        for (auto &v : adj[u]) {
29            par[v] = u;
30            dep[v] = dep[u] + 1;
31            dfs1(v);
32            sz[u] += sz[v];
33            if (sz[v] > sz[adj[u][0]]) {
34                swap(v, adj[u][0]);
35            }
36        }
37        void dfs2(int u) {
38            tin[u] = cur++;
39            seq[tin[u]] = u;
40            for (auto v : adj[u]) {
41                top[v] = v == adj[u][0] ? top[u] : v;
42                dfs2(v);
43            }
44            tout[u] = cur;
45        }
46        int lca(int u, int v) {
47            while (top[u] != top[v]) {
48                if (dep[top[u]] > dep[top[v]]) {
49                    u = par[top[u]];
50                } else {
51                    v = par[top[v]];
52                }
53            }
54            return dep[u] < dep[v] ? u : v;
55        }
56        int dist(int u, int v) { return dep[u] + dep[v] - 2 * dep[lca(u, v)]; }
57        int jump(int u, int k) {
58            if (dep[u] < k) { return -1; }
59            int d = dep[u] - k;
60            while (dep[top[u]] > d) { u = par[top[u]]; }
61            return seq[tin[u] - d];
62        }
63        // u is v's ancestor
64        bool is_ancestor(int u, int v) { return tin[u] <= tin[v] && tin[v] < tout[u]; }
65        // root's parent is itself

```

```

66    int rooted_parent(int r, int u) {
67        if (r == u) { return u; }
68        if (is_ancestor(r, u)) { return par[u]; }
69        auto it = upper_bound(adj[u].begin(), adj[u].end(), r, [&](int x, int y) {
70            return tin[x] < tin[y];
71        }) - 1;
72        return *it;
73    }
74    // rooted at u, v's subtree size
75    int rooted_size(int r, int u) {
76        if (r == u) { return n; }
77        if (is_ancestor(u, r)) { return sz[u]; }
78        return n - sz[rooted_parent(r, u)];
79    }
80    int rooted_lca(int r, int a, int b) {
81        return lca(a, b) ^ lca(a, r) ^ lca(b, r);
82    }
83
84    int main() {
85        ios::sync_with_stdio(false);
86        cin.tie(nullptr);
87
88        int n = 7;
89        HLD hld(n);
90
91        // build this tree:
92        //      0
93        //     / \
94        //    1   2
95        //   / \ / \
96        //  3  4 5  6
97
98        hld.add_edge(0, 1);
99        hld.add_edge(0, 2);
100       hld.add_edge(1, 3);
101       hld.add_edge(1, 4);
102       hld.add_edge(2, 5);
103       hld.add_edge(5, 6);
104
105       hld.build(0); // root at node 0
106
107       cout << "LCA(3,4) = " << hld.lca(3,4) << "\\n"; // expected 1
108       cout << "LCA(3,6) = " << hld.lca(3,6) << "\\n"; // expected 0
109       cout << "Distance(3,4) = " << hld.dist(3,4) << "\\n"; // expected 2
110       cout << "Distance(3,6) = " << hld.dist(3,6) << "\\n"; // expected 4
111       cout << "Is 0 ancestor of 6? " << (hld.is_ancestor(0,6) ? "yes" : "no") << "\\n"; // yes
112
113       return 0;

```

10.3 dist between u-v in tree

```

1 int dist(int a, int b) {
2     int c = lca(a, b);
3     return depth[a] + depth[b] - 2 * depth[c];
4 }

```

10.4 all longest path dfs

```

1 // all longest path (generalization of the tree diameter problem)
2 vector<tuple<int, int, int>> dp{};
3 // [mx1, x, mx2] the path of mx1 goes through x
4
5 int dfs1(int u, int p = -1) {
6     int mx = 0;
7     for (int v : adj[u]) {
8         if (v != p) {
9             int len = 1 + dfs1(v, u);
10            mx = max(mx, len);
11
12            auto& [mx1, x, mx2] = dp[u];
13            if (len >= mx1) {
14                mx2 = mx1;
15                mx1 = len;
16                x = v;
17            }
18            else if (len > mx2) {
19                mx2 = len;
20            }
21        }
22    }
23    return mx;
24 }
25
26 void dfs2(int u, int p = -1) {
27     if (p != -1) {
28         int tmx;
29         { // calculate the longest path through parent
30             auto& [mx1, x, mx2] = dp[p];
31             if (x != u) tmx = mx1 + 1;
32             else tmx = mx2 + 1;
33         }
34         { // update the path
35             auto& [mx1, x, mx2] = dp[u];
36             if (tmx >= mx1) {
37                 mx2 = mx1;
38                 mx1 = tmx;
39                 x = p;
40             }
41             else if (tmx > mx2) {
42                 mx2 = tmx;
43             }
44         }
45     }
46     for (int v : adj[u]) {
47         if (v != p) {
48             dfs2(v, u);
49         }

```

10.5 all longest path top sort

```

50 }
51 }
52
53 void all_longest_path() {
54     dp.resize(n);
55     dfs1(0), dfs2(0);
56 }

1 // all Longest path in DAG
2 // 1. topological sort
3 vector<int> in(n, 0);
4 for (int i = 0; i < m; ++i) {
5     int a, b, w;
6     cin >> a >> b >> w;
7     adj[a].emplace_back(b, w);
8     in[b]++;
9 }
10
11 vector<int> topo; // sequence of top sort
12 queue<int> q;
13 for (int i = 0; i < n; ++i) {
14     if (in[i] == 0) {
15         q.push(i);
16     }
17 }
18 while (!q.empty()) {
19     int pa = q.front();
20     q.pop();
21     topo.push_back(pa);
22     for (auto& [child, w] : adj[pa]) {
23         in[child]--;
24         if (in[child] == 0) {
25             q.push(child);
26         }
27     }
28 }
29
30 // all Longest path
31 vector<int> dist(n, INT_MIN);
32 vector<vector<int>> parents(n);
33 dist[0] = process[0];
34
35 for (int u : topo) {
36     for (auto& [v, w] : adj[u]) {
37         if (dist[v] < dist[u] + process[v] +
38             w) {
39             dist[v] = dist[u] + process[v] +
40                 w;
41             parents[v] = {u};
42         }
43         else if (dist[v] == dist[u] +
44             process[v] + w) {
45             parents[v].push_back(u);
46         }
47     }
48 }
49
50 cout << dist[n - 1];

```

10.6 Heavy-light decomposition (HLD)

```

1 const int maxn = 2e5;
2 int n, q;
3 int val[maxn];
4 int id[maxn];
5 int sz[maxn];
6 int parent[maxn];
7 int depth[maxn];
8 int tp[maxn];
9 vector<int> adj[maxn];
10 int timer = 0;

11
12 int dfs_sz(int u, int p) {
13     sz[u] = 1;
14     parent[u] = p;
15
16     // calculate size of each subtree
17     for (int v : adj[u]) {
18         if (v != p) {
19             depth[v] = depth[u] + 1;
20             sz[u] += dfs_sz(v, u);
21         }
22     }
23     return sz[u];
24 }

25 void dfs_hld(int u, int p, int top, SGT<int,
26     MergeMax>& tree) {
27     id[u] = timer++;
28     tp[u] = top;
29     tree.update(id[u], val[u]);
30
31     // find the heavy child
32     int h_v = -1, h_sz = -1;
33     for (int v : adj[u]) {
34         if (v != p) {
35             if (h_sz < sz[v]) {
36                 h_sz = sz[v];
37                 h_v = v;
38             }
39         }
40     }
41     if (h_v == -1) {
42         return;
43     }
44
45     // continue the chain
46     dfs_hld(h_v, u, top, tree);
47     // start a new chain for each light
48     // child
49     for (int v : adj[u]) {
50         if (v != p && v != h_v) {
51             dfs_hld(v, u, v, tree);
52         }
53     }
54 }

55 int path(int x, int y, SGT<int, MergeMax>&
56     tree) {
57     int ans = 0;
58     // move x and y to their top nodes until
59     // they are in the same chain

```

```

58 while (tp[x] != tp[y]) {
59     if (depth[tp[x]] < depth[tp[y]])
60         swap(x, y);
61     ans = max(ans, tree.query(id[tp[x]],
62         id[x] + 1));
63     x = parent[tp[x]];
64 }
65 if (depth[x] > depth[y]) swap(x, y);
66 ans = max(ans, tree.query(id[x], id[y] +
67     1));
68 return ans;
69 }

70 void solve() {
71     cin >> n >> q;
72     SGT<int, MergeMax> tree(n, -1);
73
74     for (int i = 0; i < n; ++i) {
75         cin >> val[i];
76     }
77     for (int i = 0; i < n - 1; ++i) {
78         int a, b;
79         cin >> a >> b;
80         a--, b--;
81         adj[a].push_back(b);
82         adj[b].push_back(a);
83     }
84
85     dfs_sz(0, 0);
86     dfs_hld(0, 0, 0, tree);
87
88     cerr << "tp: ";
89     for (int i = 0; i < n; ++i) {
90         cerr << tp[i] << " ";
91     }
92     cerr << "\n";
93
94     while (q--) {
95         int mode;
96         cin >> mode;
97         if (mode == 1) {
98             int s, x;
99             cin >> s >> x;
100             s--;
101             val[s] = x;
102             tree.update(id[s], val[s]);
103         }
104         else {
105             int a, b;
106             cin >> a >> b;
107             a--, b--;
108             cout << path(a - 1, b - 1, tree)
109                 << " ";
110         }
111     }
112 }

```

10.7 get kth ancestor

```

1 void dfs(int u, int p = -1) {
2     up[0][u] = (p == -1 ? u : p);
3     for (int v : adj[u]) {
4         if (v != p) {
5             dfs(v, u);

```

```

6         }
7     }
8 }

9 int get_kth_ancestor(int x, int k) {
10     for (int i = 0; i <= LOG; ++i) {
11         if (k & (1LL << i)) {
12             x = up[i][x];
13         }
14     }
15     return (x == 0 ? -1 : x);
16 }

17 void solve() {
18     int n, q;
19     cin >> n >> q;
20
21     adj[0].push_back(1);
22     for (int i = 2; i <= n; ++i) {
23         int u;
24         cin >> u;
25         adj[u].push_back(i);
26     }
27     dfs(0, -1);
28
29     for (int k = 1; k <= LOG; ++k) {
30         for (int v = 1; v <= n; ++v) {
31             up[k][v] = up[k - 1][up[k - 1][v]];
32         }
33     }
34
35     while (q--) {
36         int x, k;
37         cin >> x >> k;
38         cout << get_kth_ancestor(x, k) << "\n";
39     }
40 }

```

10.8 check z on path u-v

```

1 bool on_path(int z, int u, int v) {
2     return dist(u, z) + dist(z, v) == dist(u,
3         v);
}

```

10.9 path maximum edge between u, v in tree

```

1 int depth[maxn];
2 int up[LOG][maxn];
3 int maxEdge[LOG][maxn];
4
5 void dfs(int u, int p = -1, int w = 0) {
6     up[0][u] = (p == -1 ? u : p);
7     maxEdge[0][u] = (p == -1 ? 0 : w);
8     for (auto& [v, rw] : adj[u]) {

```

```

9     if (v != p) {
10         depth[v] = depth[u] + 1;
11         dfs(v, u, rw);
12     }
13 }
14 }
15
16 int lca_weight(int a, int b) {
17     if (depth[a] < depth[b]) swap(a, b);
18     int diff = depth[a] - depth[b];
19     int ans = 0;
20
21     for (int k = 0; diff; ++k) {
22         if (diff & 1) {
23             ans = max(ans, maxEdge[k][a]);
24             a = up[k][a];
25         }
26         diff >>= 1;
27     }
28     if (a == b) {
29         return ans;
30     }
31     for (int k = LOG - 1; k >= 0; --k) {
32         if (up[k][a] != up[k][b]) {
33             ans = max(ans, maxEdge[k][a]);
34             ans = max(ans, maxEdge[k][b]);
35             a = up[k][a];
36             b = up[k][b];
37         }
38     }
39
40     ans = max(ans, maxEdge[0][a]);
41     ans = max(ans, maxEdge[0][b]);
42     return ans;
43 }
44
45 void solve() {
46     dfs(0, -1, 0);
47     for (int k = 1; k < LOG; ++k) {
48         for (int v = 0; v < n; ++v) {
49             up[k][v] = up[k - 1][up[k - 1][v]];
50             maxEdge[k][v] = max(maxEdge[k - 1][v], maxEdge[k - 1][up[k - 1][v]]);
51         }
52     }
53     int path_max = lca_weight(0, 1);
54 }

```

10.10 binary jumping

```

1 const int LOG = 32;
2 const int maxn = 2e5 + 2;
3
4 int depth[maxn];
5 int up[LOG][maxn];
6 vector<int> adj[maxn];
7
8 void dfs(int u, int p = -1) {
9     up[0][u] = (p == -1 ? u : p);
10    for (int v : adj[u]) {
11        if (v != p) {

```

```

12        depth[v] = depth[u] + 1;
13        dfs(v, u);
14    }
15 }
16 }
17
18 int lca(int a, int b) {
19     if (depth[a] < depth[b]) swap(a, b);
20     int diff = depth[a] - depth[b];
21     for (int k = 0; diff; ++k) {
22         if (diff & 1) a = up[k][a];
23         diff >>= 1;
24     }
25     if (a == b) return a;
26     for (int k = LOG - 1; k >= 0; --k) {
27         if (up[k][a] != up[k][b]) {
28             a = up[k][a];
29             b = up[k][b];
30         }
31     }
32     return up[0][a];
33 }
34
35 void solve() {
36     ios::sync_with_stdio(false);
37     cin.tie(nullptr);
38
39     int n, q;
40     cin >> n >> q;
41
42     for (int i = 2; i <= n; ++i) {
43         int u;
44         cin >> u;
45         adj[u].push_back(i);
46     }
47
48     dfs(1, -1); // root = 1
49
50     for (int k = 1; k < LOG; ++k) {
51         for (int v = 1; v <= n; ++v) {
52             up[k][v] = up[k - 1][up[k - 1][v]];
53         }
54     }
55
56     while (q--) {
57         int a, b;
58         cin >> a >> b;
59         cout << lca(a, b) << "\n";
60     }
61 }

```

10.11 tree diameter

```

1 int diam = 0;
2
3 int dfs(int u, int p = -1) {
4     int mx = 0;
5     for (int v : adj[u]) {
6         if (v != p) {
7             int len = 1 + dfs(v, u);
8             diam = max(diam, mx + len);
9             mx = max(mx, len);

```

```

10     }
11 }
12 return mx;
13 }

```

10.12 all longest path

```

1 int fir[maxn]; // Length of the longest
2                 // downward path from u into its subtree.
3 int sec[maxn]; // Length of the second
4                 // longest downward path from u
5
6 void dfs1(int u, int p) {
7     for (int v : adj[u]) {
8         if (v != p) {
9             dfs1(v, u);
10            if (fir[v] + 1 > fir[u]) {
11                sec[u] = fir[u];
12                fir[u] = fir[v] + 1;
13            }
14            else if (fir[v] + 1 > sec[u]) {
15                sec[u] = fir[v] + 1;
16            }
17        }
18    }
19
20    // to_p: the best path length that comes
21    // from the parent's side
22    void dfs2(int u, int p, int to_p) {
23        res[u] = max(to_p, fir[u]);
24
25        for (int v : adj[u]) {
26            if (v != p) {
27                if (fir[v] + 1 == fir[u]) {
28                    dfs2(v, u, max(to_p, sec[u]) + 1);
29                }
30                else {
31                    dfs2(v, u, res[u] + 1);
32                }
33            }
34        }
35    }
36
37    // usage
38    dfs1(1, 0);
39    dfs2(1, 0, 0);
40    // Now res[i] is the maximum distance from
41    // node i to any other node
42    for (int i = 1; i <= n; ++i) {
43        cout << res[i] << " ";
44    }

```

10.13 tree diameter (len,end)

```

1 array<int, 2> dfs(int u, int w = -1) {
2     array<int, 2> mx{0, u}; // {length,
3     // farthest leaf}

```

```

3     for (auto& v : adj[u]) {
4         if (v == w) continue;
5         auto [len, leaf]{dfs(v, u)};
6         mx = max(mx, {len + 1, leaf});
7     }
8     return mx;
9 }
10
11 array<int, 3> tree_diameter(int a = 0) {
12     auto b{dfs(a)[1]}; // farthest node
13     // from 'a'
14     auto [1, c]{dfs(b)}; // farthest node
15     // from 'b'
16     return {1, b, c}; // {diameter
17     // length, endpoint1, endpoint2}

```

10.14 minimum distance in functional graph

```

1 // Given a functional graph (each node has
2 // exactly one outgoing edge), answer
3 // queries about the minimum distance
4 // between two nodes, or determine that
5 // it's impossible to get from one to the
6 // other.
7 // There are three cases:
8 // 1. Both in Tree
9 // 2. Both in Cycle
10 // 3. One in Tree, one in Cycle
11
12 int main() {
13     int planet_num;
14     int query_num;
15     std::cin >> planet_num >> query_num;
16     vector<int> next(planet_num);
17     vector<vector<int>> before(planet_num);
18     for (int p = 0; p < planet_num; ++p) {
19         std::cin >> next[p];
20         next[p]--;
21         before[next[p]].push_back(p);
22     }
23
24     /*
25     * -2 = We haven't even got to processing
26     * this planet yet.
27     * -1 = This node is part of a tree.
28     * >= 0: the ID of the cycle the planet
29     * belongs to.
30     */
31     vector<int> cycle_id(planet_num, -2);
32     // Each map, given a planet #, returns the
33     // index of that planet in the
34     // cycle.
35     vector<std::map<int, int>> cycles;
36     for (int p = 0; p < planet_num; ++p) {
37         if (cycle_id[p] != -2) { continue; }
38         vector<int> path{p};
39         int at = p;
40         while (cycle_id[next[at]] == -2) {
41             at = next[at];

```

```

36     cycle_id[at] = -3; // Leave
37     breadcrumbs for this iteration
38     path.push_back(at);
39 }
40 std::map<int, int> cycle;
41 bool in_cycle = false;
42 for (int i : path) {
43     in_cycle = in_cycle || i == next[at];
44     if (in_cycle) { cycle[i] = cycle.size();
45         (); }
46     cycle_id[i] = in_cycle ? cycles.size()
47         : -1;
48 }
49 cycles.push_back(cycle);
50 }
51 /*
52  * Precalculate the distance from each
53  * planet to its cycle with BFS.
54  * (cyc_dist[p] = 0) if p is part of a
55  * cycle.
56  */
57 vector<int> cyc_dist(planet_num);
58 for (int p = 0; p < planet_num; p++) {
59     // Check if this planet is part of a
60     // cycle.
61     if (cycle_id[next[p]] == -1 || cycle_id[
62         p] != -1) { continue; }
63     cyc_dist[p] = 1;
64     vector<int> stack(before[p]);
65     while (!stack.empty()) {
66         int curr = stack.back();
67         stack.pop_back();
68         cyc_dist[curr] = cyc_dist[next[curr]]
69             + 1;
70         stack.insert(stack.end(), before[curr]
71             .begin(), before[curr].end());
72     }
73 }
74 // Initialize the binary jumping arrays.
75 int log2 = std::ceil(std::log2(planet_num)
76 );
77 vector<vector<int>> pow2_ends(planet_num,
78     vector<int>(log2 + 1));
79 for (int p = 0; p < planet_num; p++) {
80     pow2_ends[p][0] = next[p]; }
81 for (int i = 1; i <= log2; i++) {
82     for (int p = 0; p < planet_num; p++) {
83         pow2_ends[p][i] = pow2_ends[pow2_ends[
84             p][i - 1]][i - 1];
85     }
86 }
87 /*
88  * Given a starting planet & dist, returns
89  * the planet we end up at
90  * if we use the teleporter dist times.
91  */
92 auto advance = [&](int pos, int dist) {
93     for (int pow = log2; pow >= 0; pow--) {
94         if ((dist & (1 << pow)) != 0) { pos =
95             pow2_ends[pos][pow]; }
96     }
97     return pos;
98 }

```

```

87 };
88 for (int q = 0; q < query_num; q++) {
89     int u, v; // going from u to v
90     std::cin >> u >> v;
91     u--;
92     v--;
93     if (cycle_id[pow2_ends[u][log2]] !=
94         cycle_id[pow2_ends[v][log2]]) {
95         cout << -1 << '\n';
96         continue;
97     }
98     if (cycle_id[u] != -1 || cycle_id[v] !=
99         -1) {
100         if (cycle_id[v] == -1 && cycle_id[u]
101             != -1) {
102             cout << -1 << '\n';
103             continue;
104         }
105         // Handle the 2nd & 3rd cases at the
106         // same time.
107         int dist = cyc_dist[u];
108         int u_cyc = advance(u, cyc_dist[u]);
109         // The root of u's tree
110         std::map<int, int> &cyc = cycles[
111             cycle_id[u_cyc]]; // u and v's
112             cycle
113         int u_ind = cyc[u_cyc];
114         int v_ind = cyc[v];
115         int diff = u_ind <= v_ind ? v_ind -
116             u_ind : cyc.size() - (u_ind -
117             v_ind);
118         cout << dist + diff << '\n';
119     } else {
120         if (cyc_dist[v] > cyc_dist[u]) {
121             cout << -1 << '\n';
122             continue;
123         }
124         int diff = cyc_dist[u] - cyc_dist[v];
125         cout << (advance(u, diff) == v ? diff
126             : -1) << '\n';
127     }
128 }
129 }
130 }

```

10.15 union length of two tree paths

```

1 /*
2 Problem: Given a tree and queries (a,b,c,d),
3 find the size of the union of nodes on
4 paths a*b and c*d.
5
6 Let P1 be the nodes on the path a*b and P2
7 the nodes on c*d.
8 We want |P1 ∩ P2| = |P1| + |P2| - |P1 ∪ P2|.
9 |P1| = dist(a,b) + 1, |P2| = dist(c,d) + 1.
10 So the only hard part is |P1 ∩ P2|.
11
12 Key facts for trees:
13
14 The intersection of two simple paths is
15 itself either empty or a simple path.

```

```

12 Any endpoint of that intersection must be
13 one of the LCAs among endpoint pairs of
14 the two paths.
15
16 Concretely the candidate nodes are
17 lca(a,b), lca(c,d), lca(a,c), lca(a,d), lca(
18 b,c), lca(b,d)
19
20 A node z lies on path u*v iff dist(u,z) +
21 dist(z,v) == dist(u,v).
22
23 So for each query:
24 compute the 6 candidate LCAs,
25 keep those candidates that lie on both paths
26 (test with distances),
27 if none → intersection size = 0; if one →
28 intersection size = 1;
29 otherwise take the two kept candidates with
30 greatest depth (they are the endpoints
31 of the intersection)
32 and intersection size = dist(p,q) + 1.
33 */
34 const int LOG = 32;
35 const int maxn = 1e5;
36 int depth[maxn];
37 vector<int> adj[maxn];
38 int up[LOG + 1][maxn];
39 int n, q;
40
41 void dfs(int u, int p = -1) {
42     up[0][u] = (p == -1 ? u : p);
43     for (int v : adj[u]) {
44         if (v != p) {
45             depth[v] = depth[u] + 1;
46             dfs(v, u);
47         }
48     }
49 }
50
51 void solve() {
52     cin >> n >> q;
53     for (int i = 0; i < n - 1; ++i) {
54         int a, b;
55         cin >> a >> b;
56         a--, b--;
57         adj[a].push_back(b);
58         adj[b].push_back(a);
59     }
60     memset(depth, 0, sizeof(depth));
61     dfs(0, -1);
62
63     for (int k = 1; k < LOG; ++k) {
64         for (int v = 0; v < n; ++v) {
65             up[k][v] = up[k - 1][up[k - 1][v]
66                 ];
67         }
68     }
69
70     while (q--) {
71         int a, b, c, d;

```

```

69     cin >> a >> b >> c >> d;
70     a--, b--, c--, d--;
71     int len1 = dist(a, b) + 1;
72     int len2 = dist(c, d) + 1;
73
74     array<int, 6> cand = {
75         lca(a, b), lca(c, d),
76         lca(a, c), lca(a, d),
77         lca(b, c), lca(b, d)
78     };
79
80     vector<int> good;
81     for (int z : cand) {
82         if (on_path(z, a, b) && on_path(
83             z, c, d)) {
84             good.push_back(z);
85         }
86     }
87
88     int inter = 0;
89     if (good.size() == 1) {
90         inter = 1;
91     } else if (good.size() > 1) {
92         sort(good.begin(), good.end(),
93             [&](int x, int y) {
94                 return depth[x] > depth[y];
95             });
96         int p = good[0], qn = good[1];
97         inter = dist(p, qn) + 1;
98     }
99     cout << (len1 + len2 - inter) << "\n
100 ";

```

10.16 Centroid decomposition

```

1 // Centroid Decomposition
2 // Problem: Given a tree with N nodes (1-
3 indexed) and M queries. Each query is
4 one of the following two types:
5 // 1 v: Paint node v red.
6 // 2 v: Print the minimal distance between
7 node v and any red node.
8
9 // Initially, only node 1 is red.
10 // Constraints: 1 ≤ N, M ≤ 10^5
11 // Time complexity: O((N + M) log N)
12
13 vector<vector<int>> adj;
14 vector<int> subtree_size;
15 // min_dist[v] = the minimal distance
16 // between v and a red node
17 vector<int> min_dist;
18 vector<bool> is_removed;
19 vector<vector<pii>> ancestors;
20
21 int get_subtree_size(int u, int p = -1) {
22     subtree_size[u] = 1;
23     for (int v : adj[u]) {
24         if (v == p || is_removed[v])
25             continue;

```

```

20     subtree_size[u] += get_subtree_size(
21         v, u);
22     }
23     return subtree_size[u];
24 }
25 int get_centroid(int u, int tree_size, int p
26     = -1) {
27     for (int v : adj[u]) {
28         if (v == p || is_removed[v])
29             continue;
30         if (subtree_size[v] * 2 > tree_size)
31             {
32                 return get_centroid(v, tree_size
33                     , u);
34             }
35     }
36     return u;
37 }
38 // Calculate the distance between current `u`
39 // and the `centroid` it belongs.
40 void get_dist(int u, int centroid, int p =
41     -1, int cur_dist = 1) {
42     for (int v : adj[u]) {
43         if (v == p || is_removed[v])
44             continue;
45         cur_dist++;
46         get_dist(v, centroid, u, cur_dist);
47         cur_dist--;
48     }
49     ancestors[u].emplace_back(centroid,
50         cur_dist);
51 }
52 void build_centroid_decomp(int u = 0) {
53     int centroid = get_centroid(u,
54         get_subtree_size(u));
55     // For all nodes in the subtree rooted
56     // at `centroid`, calculate their
57     // distance to the centroid
58     for (int v : adj[centroid]) {
59         if (is_removed[v]) continue;
60         get_dist(v, centroid, centroid);
61     }
62     is_removed[centroid] = true;
63     for (int v : adj[centroid]) {
64         if (is_removed[v]) continue;
65         build_centroid_decomp(v);
66     }
67 }
68 // Paint `node` red by updating all of its
69 // ancestors' minimal distances to a red
70 // node
71 void paint(int u) {
72     for (auto &[ancestor, dist] : ancestors[
73         u]) {
74         min_dist[ancestor] = min(min_dist[
75             ancestor], dist);
76     }
77     min_dist[u] = 0;
78 }
79 }
80 // Print the minimal distance between `u` to
81 // a red node.
82 void query(int u) {
83     int ans = min_dist[u];
84     for (auto &[ancestor, dist] : ancestors[
85         u]) {
86         if (!dist) continue;
87         ans = min(ans, dist + min_dist[
88             ancestor]);
89     }
90     cout << ans << "\n";
91 }
92 void solve() {
93     int N, M;
94     cin >> N >> M;
95     adj.assign(N, vector<int>());
96     for (int i = 0; i < N - 1; ++i) {
97         int a, b;
98         cin >> a >> b;
99         a--, b--;
100         adj[a].push_back(b);
101         adj[b].push_back(a);
102     }
103     subtree_size.assign(N, 0);
104     ancestors.assign(N, vector<pii>());
105     is_removed.assign(N, false);
106     build_centroid_decomp();
107     min_dist.assign(N, INF);
108     paint(0);
109     for (int i = 0; i < M; ++i) {
110         int t, v;
111         cin >> t >> v;
112         v--;
113         if (t == 1) {
114             paint(v);
115         }
116         else {
117             query(v);
118         }
119     }
120 }

```

11 default

11.1 debug

```

1 #ifndef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr, "%s - %d : (%s) = ",
4         __PRETTY_FUNCTION__, __LINE__, #
5         __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8 template<typename I> void _DO(I&&x){cerr<<x
9     <<endl;}

```

```

7 template<typename I, typename...T> void _DO(I
8     &&x, T&&...tail){cerr<<x<<" "; _DO(tail
9     ...);}
10 #else
11 #define dbg(...)
12 #endif

```

11.2 template

```

1 // alias g++='g++ -std=c++14 -fsanitize=
2 // undefined -Wall -Wextra -Wshadow -D
3 // LOCAL'
4 #include <bits/stdc++.h>
5 using namespace std;
6 #ifdef LOCAL
7 #include "../debug.h"
8 #else
9 #pragma GCC optimize("O3,unroll-loops")
10 #pragma GCC target("avx2,bmi,bmi2,lzcnt,
11     popcnt")
12 #define debug(...) ((void)0)
13 #define orange(...) ((void)0)
14 #endif
15 #define int long long
16 #define pii pair<int, int>
17 #define ff first
18 #define ss second
19 #define pb push_back
20 #define SPEEDY ios_base::sync_with_stdio(
21     false); cin.tie(0); cout.tie(0);
22 void solve() {
23 }
24 }
25 signed main() {
26     SPEEDY;
27     return 0;
28 }

```

11.3 vimrc

```

1 set nu
2 set ts=4
3 set shiftwidth=4
4 set autoindent smartindent
5 set cindent
6
7 imap jk <ESC>
8 inoremap {<CR> {<CR><ESC>ko

```

12 other

12.1 Nim game

```
1 | a1^a2^a3^...^an != 0 ? A win : B win

```

12.2 找小於 n 所有出現的 1 數量

```

1 | current == 0 higher * factor
2 | current == 1 higher * factor + lower + 1
3 | other current (higher + 1) * factor

```

13 other language

13.1 python heap

```

1 | import heapq
2 |
3 | heap = [7,1,2,2]
4 | heapq.heapify(heap)
5 | print(heap) # [1, 2, 2, 7]
6 | heapq.heappush(heap, 5)
7 | print(heap) # [1, 2, 2, 7, 5]
8 | print(heapq.heappop(heap)) # 1
9 | print(heap) # [2, 2, 5, 7]

```

13.2 java

13.2.1 文件操作

```

1 | import java.io.*;
2 | import java.util.*;
3 | import java.math.*;
4 | import java.text.*;
5 |
6 | public class Main{
7 |
8 |     public static void main(String args[]){
9 |         throws FileNotFoundException,
10 |             IOException
11 |         Scanner sc = new Scanner(new FileReader(
12 |             "a.in"));
13 |         PrintWriter pw = new PrintWriter(new
14 |             FileWriter("a.out"));
15 |         int n,m;
16 |         n=sc.nextInt();//读入下一个INT
17 |         m=sc.nextInt();
18 |
19 |         for(ci=1; ci<=c; ++ci){
20 |             pw.println("Case #"+ci+": easy for
21 |                 output");
22 |         }
23 |     }
24 | }

```



```

17 }
18
19 pw.close();//关闭流并释放 · 这个很重要 ·
    否则是没有输出的
20 sc.close();//关闭流并释放
21 }
22 }

```

13.2.2 优先队列

```

1 PriorityQueue queue = new PriorityQueue( 1,
    new Comparator(){
2     public int compare( Point a, Point b ){
3         if( a.x < b.x || a.x == b.x && a.y < b.y )
4             return -1;
5         else if( a.x == b.x && a.y == b.y )
6             return 0;
7         else return 1;
8     }
9 });

```

13.2.3 Map

```

1 Map map = new HashMap();
2 map.put("sa", "dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj);
7 }

```

13.2.4 sort

```

1 static class cmp implements Comparator{
2     public int compare(Object o1, Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8 public static void main(String[] args)
9     throws IOException{
10     Scanner cin = new Scanner(System.in);
11     int n;
12     n=cin.nextInt();
13     BigInteger[] seg = new BigInteger[n];
14     for (int i=0;i<n;i++)
15         seg[i]=cin.nextBigInteger();
16     Arrays.sort(seg, new cmp());
17 }

```

13.3 python output

```

1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB {}'.format('Jin', 'Kela'))
10 # AAA Jin BBB "Kela!"
11
12 hello = 'hello, world\n'
13 hellos = repr(hello)
14 print(hellos) # 'hello, world\n'
15
16 x = 32.5
17 y = 40000
18 print(repr((x, y, ('spam', 'eggs'))))
19 # "(32.5, 40000, ('spam', 'eggs'))"
20
21 x = 7
22 print(eval('3 * x')) # 21

```

13.4 python 大數因數分解

```

1 # 大數因數分解 (使用 Pollard's Rho 與 Miller
    -Rabin)
2 import sys, random
3 from math import gcd
4
5 # Miller-Rabin 檢定 (機率性質數判定)
6 def is_probable_prime(n, k=12):
7     if n < 2:
8         return False
9     # 先檢查一些小質數
10    small_primes =
11    [2,3,5,7,11,13,17,19,23,29]
12    for p in small_primes:
13        if n % p == 0:
14            return n == p
15    # 把 n-1 寫成 d * 2^s
16    d = n - 1
17    s = 0
18    while d % 2 == 0:
19        d //= 2
20        s += 1
21    # 重複 k 次隨機測試
22    for _ in range(k):
23        a = random.randrange(2, n - 1) # 隨
24        # 機挑一個測試基數
25        x = pow(a, d, n)
26        if x == 1 or x == n - 1:
27            continue
28        composite = True
29        for __ in range(s - 1):
30            x = pow(x, 2, n)
31            if x == n - 1:
32                composite = False
33                break
34        if composite:
35            return False

```

```

34 return True
35
36 # Pollard's Rho 演算法 (找非平凡因數)
37 def pollards_rho(n):
38     if n % 2 == 0:
39         return 2
40     if n % 3 == 0:
41         return 3
42     # 隨機多項式 (x^2 + c) mod n
43     while True:
44         c = random.randrange(1, n-1) #
45         # 隨機挑選常數 c
46         x = random.randrange(2, n-1) #
47         # 起始點
48         y = x
49         d = 1
50         while d == 1:
51             x = (pow(x, 2, n) + c) % n # x
52             # -> f(x)
53             y = (pow(y, 2, n) + c) % n # y
54             # -> f(f(y)) · 走兩步
55             y = (pow(y, 2, n) + c) % n #
56             # -> f(f(y)) · 走兩步
57             d = gcd(abs(x - y), n) #
58             # 計算兩者差的 gcd
59             if d == n:
60                 # 失敗就重試
61                 break
62             if d > 1 and d < n:
63                 # 找到非平凡因數
64                 return d
65
66 # 遞迴分解
67 def factor(n, out):
68     if n == 1:
69         return
70     if is_probable_prime(n):
71         out.append(n)
72     else:
73         d = pollards_rho(n)
74         while d is None or d == n: # 偶爾失
75             # 敗就重試
76             d = pollards_rho(n)
77         factor(d, out)
78         factor(n // d, out)
79
80 def main():
81     data = sys.stdin.read().strip().split()
82     if not data:
83         return
84     # 每個 token 當作一個數字
85     for token in data:
86         try:
87             n = int(token)
88             except:
89                 continue
90             if n <= 1:
91                 print(n)
92                 continue
93             facs = []
94             factor(n, facs)
95             facs.sort()
96             # 輸出因數
97             print(" ".join(str(x) for x in facs)
98 )

```

```

89
90 if __name__ == "__main__":
91     random.seed() # 使用系統時間作為隨機種
92     子
93     main()

```

13.5 decimal

```

1 # 使用 decimal 模組來處理高精度小數運算
2 from decimal import *
3 setcontext(Context(prec=MAX_PREC, Emax=
4     MAX_EMAX, rounding=ROUND_FLOOR))
5 print(Decimal(input()) * Decimal(input()))
6
7 # 將小數轉成分數 · 方便做近似或理論分析 · 且可
8 # 以限制分母大小 ·
9 from fractions import Fraction
10 Fraction('3.14159').limit_denominator(10).
11     numerator # 22
12
13 # 設定精確度
14 from decimal import Decimal, getcontext
15
16 # 精確位數設定
17 getcontext().prec = 70
18
19 n = 100
20 # 指定 n 為高精確度的物件
21 n = Decimal(n)
22 n /= 7
23 print(n)
24
25 # 將小數轉成分數
26 from fractions import Fraction
27
28 n = 1.5654
29 # 建立一個轉換物件
30 n = Fraction(n)
31
32 print(n)

```

13.6 python 大數排序

```

1 # 大數排序
2 # Line one n : 多少數字
3 # next line : 依序輸入每行一個
4 # sort : sort + lambda
5 from sys import stdin
6
7 data = stdin.read().splitlines()
8
9 i = 0
10
11 while(i < len(data)):
12     T = int(data[i].strip())
13     i += 1
14     temp = [int(data[i + index].strip()) for
15         index in range(T)]

```

```

15 | i += T
16 | temp = sorted(temp, key = lambda x : (x)
17 | )
18 | for ele in temp:
    | print(ele)

```

13.7 python 大數計算 2

```

1 | # 單行輸入
2 | # format : n1, operation, n2
3 | from sys import stdin
4 |
5 | data = stdin.read().splitlines()
6 |
7 | limit = len(data)
8 | i = 0
9 |
10 | while(i < limit):
11 |     a, operation, b = map(str, data[i].split
12 |         ())
13 |     a, b = int(a), int(b)
14 |     i += 1
15 |     if(operation == '+'):
16 |         print(int(a + b))
17 |     elif(operation == '-'):
18 |         print(int(a - b))
19 |     elif(operation == '*'):
20 |         print(int(a * b))
21 |     else:
22 |         print(int(a // b))

```

13.8 python input

```

1 | ans = sum(map(float, input().split()))
2 | # input: 1.1 2.2 3.3 4.4 5.5
3 | print(ans) # 16.5
4 |
5 | (n, m) = map(int, input().split()) # 300 200
6 | print(n * m) # 60000
7 |
8 | Arr = list(map(int, input().split()))
9 | # input: 1 2 3 4 5
10 | print(Arr) # [1, 2, 3, 4, 5]

```

13.9 python 大數計算

```

1 | # 讀取多行輸入
2 | # line one first number
3 | # line two operation
4 | # line three second number
5 | from sys import stdin
6 |
7 | data = stdin.read().splitlines()
8 |
9 | limit = len(data)
10 | i = 0

```

```

11 | while(i < limit):
12 |     a = int(data[i].strip())
13 |     i += 1
14 |     operation = data[i].strip()
15 |     i += 1
16 |     b = int(data[i].strip())
17 |     i += 1
18 |     if(operation == '*'):
19 |         print(int(a * b))
20 |     else:
21 |         print(int(a / b))

```

13.10 base-6 pow

```

1 | import sys
2 |
3 | sys.setrecursionlimit(2000000)
4 |
5 | def solve():
6 |     S = sys.stdin.readline().strip()
7 |     try:
8 |         N = int(S)
9 |     except ValueError:
10 |         N = 0
11 |     if N == 0:
12 |         print(1)
13 |         return
14 |     low = 1
15 |     high = 700000 # Safely larger than
16 |         500000 * 1.3
17 |     ans = high + 1
18 |     while low <= high:
19 |         L_mid = (low + high) // 2
20 |         if L_mid <= 0: # Avoid 6^0 or 6^
21 |             negative
22 |             low = L_mid + 1
23 |             continue
24 |         val = pow(6, L_mid)
25 |         if val > N:
26 |             ans = L_mid
27 |             high = L_mid - 1
28 |         else:
29 |             low = L_mid + 1
30 |     print(ans)
31 | solve()

```

14 zformula

14.1 formula

14.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

14.1.2 圖論

- 對於平面圖 $F = E - V + C + 1$ ， C 是連通分量數
- 對於平面圖 $E < 3V - 6$
- 對於連通圖 G ，最大獨立點集的大小設為 $I(G)$ ，最大匹配大小設為 $M(G)$ ，最小點覆蓋設為 $Cv(G)$ ，最小邊覆蓋設為 $Ce(G)$ 。對於任意連通圖：

$$(a) I(G) + Cv(G) = |V|$$

$$(b) M(G) + Ce(G) = |V|$$

- 對於連通二分圖：

$$(a) I(G) = Cv(G)$$

$$(b) M(G) = Ce(G)$$

14.1.3 學長公式

- $\sum_{d|n} \phi(n) = n$
- Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
- $\gamma = 0.57721566490153286060651209008240243104215$
- 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

14.1.4 基本數論

- $\sum_{d|n} \mu(n) = [n == 1]$
- $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
- $\sum_{i=1}^n \sum_{j=1}^m$ 互質數量 $= \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
- $\sum_{i=1}^n \sum_{j=1}^m lcm(i, j) = n \sum_{d|n} d \times \phi(d)$

14.1.5 排組公式

- k 卡特蘭 $\frac{C_n^{k,n}}{n(k-1)+1} \cdot C_n^m = \frac{n!}{m!(n-m)!}$
- $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
- Stirling number of $2^n d, n$ 人分 k 組方法數目
 - $S(0, 0) = S(n, n) = 1$
 - $S(n, 0) = 0$
 - $S(n, k) = kS(n-1, k) + S(n-1, k-1)$
- Bell number, n 人分任意多組方法數目
 - $B_0 = 1$
 - $B_n = \sum_{i=0}^n S(n, i)$
 - $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
 - $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
 - $B_{p+m+n} \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
 - From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$

- Derangement, 錯排，沒有人在自己位置上
 - $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
 - $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
 - From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

6. Binomial Equality

$$(a) \sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n}$$

$$(b) \sum_k \binom{l}{m+k} \binom{s}{n+k} = \binom{l+s}{l-m+n}$$

$$(c) \sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$$

$$(d) \sum_{k \leq l} \binom{l-k}{m} \binom{s}{k-n} (-1)^k = \frac{(-1)^{l+m} \binom{s-m-1}{l-n-m}}{(-1)^{l+m} \binom{s-m-1}{l-n-m}}$$

$$(e) \sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$$

$$(f) \binom{l}{k} = (-1)^k \binom{k-r-1}{k}$$

$$(g) \binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$$

$$(h) \sum_{k \leq n} \binom{r+k}{m} = \binom{r+n+1}{m+1}$$

$$(i) \sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$$

$$(j) \sum_{k \leq m} \binom{m+r}{k} x^k y^{m-k} = \sum_{k \leq m} \binom{-r}{k} (-x)^k (x+y)^{m-k}$$

14.1.6 冪次, 冪次和

- $a^{b\%p} P = a^{b\% \varphi(p) + \varphi(p)}, b \geq \varphi(p)$
- $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
- $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
- $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
- $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
- $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
- $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
- 除了 $B_1 = -1/2$ ，剩下的奇數項都是 0
- $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

14.1.7 循環小數轉分數

- 若 $x = 0.\bar{a}$ ，則

$$x = \frac{a}{\overbrace{99 \dots 9}^{k \text{ digits}}}$$

其中 a 為循環節， k 為循環節的位數。

- 例子：

$$0.\overline{37} = \frac{37}{99}$$

$$0.\overline{5} = \frac{5}{9}$$

14.1.8 循環小數轉分數

1. 純循環小數：若 $x = 0.\overline{a}$ ，其中 a 為循環節、長度為 k 。

$$x = \underbrace{\frac{a}{99 \dots 9}}_{k \text{ digits}}$$

例：

$$0.\overline{37} = \frac{37}{99}, \quad 0.\overline{5} = \frac{5}{9}$$

2. 混循環小數：若 $x = 0.b\overline{a}$ ，其中 b 為前綴、長度 m ， a 為循環節、長度 k 。

$$x = \frac{(b \cdot 10^k + a) - b}{10^m(10^k - 1)}$$

例：

$$0.12\overline{3} = \frac{(12 \cdot 10^1 + 3) - 12}{10^2(10^1 - 1)} = \frac{123 - 12}{100 \cdot 9} = \frac{111}{900} = \frac{37}{300}$$

14.1.9 常見級數與組合公式

1. 平方和公式：

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$4^2 + 6^2 + \dots + (2n)^2 = \frac{(2n)(n+1)(2n+1)}{3}$$

$$1^2 + 3^2 + \dots + (2n+1)^2 = \frac{n(2n-1)(2n+1)}{3}$$

2. 立方和公式：

$$1^3 + 2^3 + \dots + n^3 = \frac{n^4 + 2n^3 + n^2}{4}$$

3. 四次方和公式：

$$1^4 + 2^4 + \dots + n^4 = \frac{6n^5 + 15n^4 + 10n^3 - n}{30}$$

4. 五次方和公式：

$$1^5 + 2^5 + \dots + n^5 = \frac{2n^6 + 6n^5 + 5n^4 - n^2}{12}$$

5. 六次方和公式：

$$1^6 + 2^6 + \dots + n^6 = \frac{6n^7 + 21n^6 + 21n^5 - 7n^3 + n}{42}$$

6. 七次方和公式：

$$1^7 + 2^7 + \dots + n^7 = \frac{3n^8 + 12n^7 + 14n^6 - 7n^4 + 2n^2}{24}$$

7. 八次方和公式：

$$\begin{aligned} & 1^8 + 2^8 + \dots + n^8 \\ &= \frac{10n^9 + 45n^8 + 60n^7 - 42n^5 + 20n^3 - 3n}{90} \end{aligned}$$

8. 九次方和公式：

$$\begin{aligned} & 1^9 + 2^9 + \dots + n^9 \\ &= \frac{2n^{10} + 10n^9 + 15n^8 - 14n^6 + 10n^4 - 3n^2}{20} \end{aligned}$$

9. 十次方和公式：

$$\begin{aligned} & 1^{10} + 2^{10} + \dots + n^{10} \\ &= \frac{6n^{11} + 33n^{10} + 55n^9 - 66n^7 + 66n^5 - 33n^3 + 5n}{66} \end{aligned}$$

10. 等比級數：

$$S = a \cdot \frac{r^n - 1}{r - 1}$$

11. 二項式係數恆等式：

$$\binom{n}{0} + \binom{n}{1} + \dots + \binom{n}{n} = 2^n$$

$$\binom{n}{1} + \binom{n}{2} + \dots + \binom{n}{n} = 2^n - 1$$

$$\binom{n}{1} + \binom{n}{3} + \dots + \binom{n}{n-k} = 2^{n-1}$$

12. 分配問題（玩具分給小孩）：

- (a) n 個玩具， k 位小孩，可以有人沒拿到：

$$\binom{n+k-1}{n} = \binom{n+k-1}{k-1}$$

- (b) n 個玩具， k 位小孩，每個人至少一個：

$$\binom{n-1}{k-1}$$

14.1.10 位元運算

- (a) 位元條件：

$$(x+k) \& (y+k) = 0$$

- (b) 加法恆等式（利用 XOR 與 AND）：

$$a+b = (a \oplus b) + 2 \cdot (a \& b)$$

- (c) OR 與 AND 的關係：

$$a \mid b = a + b - (a \& b)$$

- (d) 交換兩數：

$$a = a \oplus b, \quad b = a \oplus b, \quad a = a \oplus b$$

- (e) 取得最低位的 1：

$$x \& (-x)$$

- (f) 清除最低位的 1：

$$x \& (x-1)$$

14.1.11 除數與倍數

- (a) LCM:

$$\text{lcm}(a, b) = \frac{a \times b}{\text{gcd}(a, b)}$$

- (b) LCM (prime factorization):

$$\text{lcm}(a, b) = \prod_{p \in \text{primes}} p^{\max(e_a, e_b)}$$

- (c) GCD (prime factorization):

$$\text{gcd}(a, b) = \prod_{p \in \text{primes}} p^{\min(e_a, e_b)}$$

- (d) Counting divisors:

$$\text{If } n = \prod_{i=1}^k p_i^{e_i}, \text{ cnt is } \prod_{i=1}^k (e_i + 1)$$

14.1.12 數論公式

13. Bezout's identity：

$$ax + by = \text{gcd}(a, b) \quad (\text{必定存在整數解 } x, y)$$

14. 模指數運算（冪的冪）：

$$a^{b^c} \equiv a^{\text{power_mod}(b, c, \text{MOD}-1)} \pmod{\text{MOD}}$$

Pythagorean theorem：

$$(n^2 + m^2, 2nm, n^2 - m^2), \quad n > m > 0$$

Wilson's theorem：

$$(p-1)! \pmod p = p-1 \quad (p \text{ 為質數})$$

Catalan number：

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

海龍公式：

$$\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2}$$

兩圓相交弦長：

$$\text{Length} = 2 \cdot \sqrt{\frac{(s-a)(s-b)}{s}}, \quad s = \frac{r_1 + r_2 + d}{2}$$

三角形內切圓半徑：

$$r = \frac{2 \cdot \text{Area}}{a+b+c}$$

三角形外接圓半徑：

$$R = \frac{abc}{4 \cdot \text{Area}}$$

三角形重心：

$$G = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3} \right)$$

三角形垂心：

$$H = \left(\frac{x_1 \tan A + x_2 \tan B + x_3 \tan C}{\tan A + \tan B + \tan C}, \frac{y_1 \tan A + y_2 \tan B + y_3 \tan C}{\tan A + \tan B + \tan C} \right)$$

三角形內心：

$$I = \left(\frac{ax_1 + bx_2 + cx_3}{a+b+c}, \frac{ay_1 + by_2 + cy_3}{a+b+c} \right)$$

奇(偶)長度陣列子數量：

$$n = \frac{(i+1) * (n-i) + 1}{2}$$

15 Интернационал

15.1 保佑

[illegible]

34 #
35 #
36 #
37 #
38 #
39 #
40 #
41 #
42 #
43 #
44 #
45 #
46 #
47 #
48 #
49 #
50 #
51 #
52 #
53 #
54 #

神獸保佑 永無BUG!

```

56 // ## #####
57 // ## ##
58 // ## ##
59 // ## ##
60 // ## ##
61 // ## ##
62 // ## ##
63 // ## ##
64 // ## ##
65 // ## ##
66 // #####
67 // ## ##
68 // ## ##
69 // ## ##
70 // ## ##
71 // ## ##
72 // ## ##
73 // ## ##
74 // ##### ##
75 //
76 // 元首保佑 永無BUG
77
78 //
79 // <_@ \
80 // \(_)\ \**/ \(_)/ \
81 // \(_)\ \**/ \(_)/ \
82 // \(_)\ \**/ \(_)/ \
83 // u/u/u/****/u\u\u
84 // u/u/****/u\u
85 // |*|*|
86 // | |
87 // /+--+ \
88 // || 卐 ||
89 // \ |=/
90 //
91 // 神獸保佑 永無BUG

```

ACM ICPC Team Reference - BogoSort

Contents

1 Algorithm	1	4 Data Structure	5	7 Number Theory	13	10 Tree	18
1.1 LIS	1	4.1 undo disjoint set	5	7.1 Linear Sieve	13	10.1 Euler tour+RMQ	18
1.2 LCS	1	4.2 lazy propagation	5	7.2 C(n,m)	13	10.2 HLD template	19
2 Basic	1	4.3 Lazy heap	6	7.3 Euler Totient 1 to n	13	10.3 dist between u-v in tree	19
2.1 mod helper function	1	4.4 Fenwick tree (BIT)	6	7.4 derangement (Principle of Inclusion-Exclusion)	14	10.4 all longest path dfs	19
2.2 self-defined-pq-operator	1	4.5 BIT - range update + range prefix sum	6	7.5 matrix template (with fast power)	14	10.5 all longest path top sort	20
2.3 generating all subsets	1	4.6 Order Statistics Tree	6	7.6 Sieve of Eratosthenes (with big num)	14	10.6 Heavy-light decomposition (HLD)	20
2.4 tuple	1	4.7 Trie (Prefix tree)	6	7.7 mod inv	14	10.7 get kth ancestor	20
2.5 memset	1	4.8 segment tree	6	7.8 derangement (DP)	14	10.8 check z on path u-v	20
2.6 submask enumeration	1	4.9 BIT range update point query	7	7.9 Euler Totient	14	10.9 path maximum edge between u, v in tree	20
2.7 stringstream split by comma	1	4.10 DSU	7	7.10 fast power	14	10.10 binary jumping	21
3 DP	1	5 Geometry	7	7.11 first and second mex	14	10.11 tree diameter	21
3.1 deque	1	5.1 cross. product	7	7.12 Catalan Number	14	10.12 all longest path	21
3.2 walk	1	5.2 Check Point in Polygon	7	7.13 Chinese Remainder Theorem	14	10.13 tree diameter (len,end)	21
3.3 grouping	2	5.3 Point	8	7.14 mod inv (not prime)	15	10.14 minimum distance in functional graph	21
3.4 matching	2	5.4 Polygon Lattice Points	8	7.15 mod inv (not coprime)	15	10.15 union length of two tree paths	22
3.5 projects	2	5.5 line intersect	8	7.16 inclusion-exclusion principle	15	10.16 Centroid decomposition	22
3.6 stones	2	5.6 Polygon area	8	7.17 C(n,k) mod inverse	15	11 default	23
3.7 coins	2	5.7 using std::complex	8	7.18 Linear Diophantine 丢番圖	15	11.1 debug	23
3.8 elevator rides	3	5.8 point distance from a line	9	7.19 擴展歐基里德	15	11.2 template	23
3.9 slimes	3	6 Graph	9	7.20 Sieve of Eratosthenes	15	11.3 vimrc	23
3.10 digit sum	3	6.1 Prim	9	7.21 builtin	15	12 other	23
3.11 sushi	3	6.2 Eulerian cycle	9	7.22 C(n,k) DP	16	12.1 Nim game	23
3.12 candies	3	6.3 maximum weight bipartite matching	9	8 Range Queries	16	12.2 找小於 n 所有出現的 1 數量	23
3.13 permutation	4	6.4 maximum flow	10	8.1 subarray maximum sum queries	16	13 other language	23
3.14 Knasack2	4	6.5 maximum cardinality bipartite matching	11	8.2 find first equal or greater	16	13.1 python heap	23
3.15 flowers	4	6.6 Floyd-Warshall	11	8.3 find k-th and count less than	16	13.2 java	23
3.16 independent set	4	6.7 MST	11	9 String	17	13.2.1 文件操作	23
3.17 counting tower	4	6.8 Hopcroft-Karp	11	9.1 Z	17	13.2.2 优先队列	24
3.18 counting numbers	5	6.9 Dijkstra	11	9.2 manacher	17	13.2.3 Map	24
		6.10 Dinic	11	9.3 AC 自動機	17	13.2.4 sort	24
		6.11 maximum cardinality matching	12	9.4 KMP	18	13.3 python output	24
		6.12 topological sort	12	9.5 suffix array lcp	18	13.4 python 大數因數分解	24
		6.13 Floyd's algorithm	12	9.6 hash	18	13.5 decimal	24
		6.14 stable matching	13	9.7 minimal string rotation	18	13.6 python 大數排序	24
		6.15 Bellman-Ford	13	9.8 reverseBWT	18	13.7 python 大數計算 2	25
						13.8 python input	25
						13.9 python 大數計算	25
						13.10 base-6 pow	25

14	zformula	25	14.1.3	學長公式	25	14.1.7	循環小數轉分數	25	14.1.11	除數與倍數	26		
	14.1	formula	25	14.1.4	基本數論	25	14.1.8	循環小數轉分數	26	14.1.12	數論公式	26	
		14.1.1	Pick 公式	25	14.1.5	排組公式	25	14.1.9	常見級數與組合公式 .	26	15	Интернационал	27
		14.1.2	圖論	25	14.1.6	冪次, 冪次和	25	14.1.10	位元運算	26	15.1	保佑	27

ACM ICPC Judge Test - BogoSort

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6     const size_t KB = 1024;
7     const size_t MB = KB * 1024;
8     const size_t GB = MB * 1024;
9
10    size_t block_size, bound;
11    void stack_size_dfs(size_t depth = 1) {

```

```

12        if (depth >= bound)
13            return;
14        int8_t ptr[block_size]; // 若無法編譯將
15            block_size 改成常數
16        memset(ptr, 'a', block_size);
17        cout << depth << endl;
18        stack_size_dfs(depth + 1);
19    }
20    void stack_size_and_runtime_error(size_t
21        block_size, size_t bound = 1024) {
22        system_test::block_size = block_size;
23        system_test::bound = bound;
24        stack_size_dfs();
25    }
26    double speed(int iter_num) {
27        const int block_size = 1024;
28        volatile int A[block_size];
29        auto begin = chrono::high_resolution_clock
30            ::now();
31        while (iter_num--)
32            for (int j = 0; j < block_size; ++j)
33                A[j] += j;
34        auto end = chrono::high_resolution_clock::
35            now();
36        chrono::duration<double> diff = end -
37            begin;

```

```

38        return diff.count();
39    }
40    void runtime_error_1() {
41        // Segmentation fault
42        int *ptr = nullptr;
43        *(ptr + 7122) = 7122;
44    }
45    void runtime_error_2() {
46        // Segmentation fault
47        int *ptr = (int *)memset;
48        *ptr = 7122;
49    }
50    void runtime_error_3() {
51        // munmap_chunk(): invalid pointer
52        int *ptr = (int *)memset;
53        delete ptr;
54    }
55    void runtime_error_4() {
56        // free(): invalid pointer
57        int *ptr = new int[7122];
58        ptr += 1;
59        delete[] ptr;
60    }
61    }
62

```

```

63    void runtime_error_5() {
64        // maybe illegal instruction
65        int a = 7122, b = 0;
66        cout << (a / b) << endl;
67    }
68    void runtime_error_6() {
69        // floating point exception
70        volatile int a = 7122, b = 0;
71        cout << (a / b) << endl;
72    }
73    void runtime_error_7() {
74        // call to abort.
75        assert(false);
76    }
77    } // namespace system_test
78
79    #include <sys/resource.h>
80    void print_stack_limit() { // only work in
81        Linux
82        struct rlimit l;
83        getrlimit(RLIMIT_STACK, &l);
84        cout << "stack_size = " << l.rlim_cur << "
85            byte" << endl;
86    }
87

```